



Certification professionnelle

Titre professionnel: **Concepteur développeur d'applications**

RNCP: **37873**

Dossier Projet

Projet : M&C Billetterie

Rédacteur : Coulet Corentin

Date : 27/06/2025

1. Liste des compétences du référentiel couvertes par le projet

Le projet M&C Society permet de mobiliser l'ensemble des compétences attendues du titre professionnel "Concepteur Développeur d'Applications" (RNCP 37873).

Bloc 1 - Concevoir et développer des composants d'interface utilisateur en intégrant les recommandations de sécurité

Le développement du front-end en Next.js a permis de concevoir et d'implémenter une interface utilisateur claire, moderne et responsive. La charte graphique bleue, noir et blanche a été respectée afin d'assurer la cohérence visuelle du produit.

Des bonnes pratiques d'accessibilité ont été intégrées, notamment l'utilisation de contrastes adaptés, la navigation clavier et l'ajout de textes alternatifs sur les images.

La sécurisation des échanges a été prise en compte dès l'authentification, via la gestion des sessions utilisateurs et l'intégration de mécanismes de hachage (bcrypt) et de tokens (JWT).

Bloc 2 - Concevoir et développer la persistance des données en intégrant les recommandations de sécurité

La modélisation des données a été réalisée à l'aide de **PlantUML** pour définir un schéma Entité-Relation, avant d'être implémentée en **PostgreSQL** via l'ORM **Prisma**.

Le système gère la persistance des utilisateurs, des rôles, des événements, des billets et des commandes. Le projet intègre également les bonnes pratiques liées au RGPD : respect du droit à l'oubli, possibilité d'exporter ou de supprimer les données personnelles, ainsi que la sécurisation des accès aux données sensibles.

La sécurité des données est renforcée par l'utilisation de l'ORM, réduisant le risque d'injections SQL, et par la sanitisation des données utilisateurs.

Bloc 3 - Concevoir et développer une application multicouche répartie en intégrant les recommandations de sécurité

Le projet repose sur une architecture modulaire en couches hybride, adaptée à **Next.js 13+ App Router**, séparant clairement la logique métier (services spécialisés), l'accès aux données (repository via Prisma ORM) et les contrôleurs exposés via un adaptateur vers les API routes. Cette organisation assure une excellente **maintenabilité, testabilité et évolutivité**. L'intégration de Stripe pour les paiements et d'un système d'email transactionnel garantit une expérience utilisateur complète. Le système de rôles granulaire (USER/ORGANIZER/ADMIN) avec middleware **RBAC** assure une sécurité robuste et une gestion fine des accès.

2. Cahier des charges

2.1. Description de l'existant

La gestion d'événements implique souvent l'utilisation de plusieurs outils distincts pour les réservations, la billetterie, le suivi commercial et la relation client. Cette fragmentation peut nuire à la fluidité des opérations, ralentir la prise de décision et dégrader l'expérience des utilisateurs.

Le projet M&C Society répond à ce constat en proposant une solution centralisée et intuitive, pensée pour simplifier l'organisation d'événements tout en offrant aux utilisateurs finaux une expérience fluide et cohérente. L'objectif est de permettre aux organisateurs de gagner en autonomie, en efficacité et en visibilité sur leurs performances.

2.2. Reprise de l'existant

Il n'existe aucune version antérieure du projet. Il s'agit d'une création ex nihilo. Aucun nom de domaine ni hébergement n'est encore attribué. Aucun patrimoine documentaire n'est à reprendre : ni ancien cahier des charges, ni spécification fonctionnelle.

2.3. Principes de référencement

Le projet vise un bon positionnement naturel dans les moteurs de recherche (SEO). L'objectif est de maximiser la visibilité des événements publics, tout en assurant un référencement clair des pages événementielles. Le recours aux balises structurées, à un plan de site dynamique, ainsi qu'à des URL claires et optimisées est prévu.

2.4. Exigences de performances et de volumétrie

La plateforme vise à accueillir jusqu'à 10 000 visiteurs uniques par mois en phase de lancement, avec une montée en charge progressive jusqu'à 50 000 utilisateurs mensuels. Le temps de réponse moyen attendu est inférieur à 300 millisecondes pour les actions courantes (navigation, consultation d'un événement). Une disponibilité de 99,5 % est exigée. Le catalogue pourra regrouper jusqu'à 1 000 événements actifs simultanément, avec des pics de consultation à prévoir lors des lancements ou campagnes de communication.

2.5. Multilinguisme & adaptations pour un public spécifique

Le site sera initialement disponible en français. Une ouverture à l'anglais est prévue dans une future phase. Le contenu sera géré de manière à faciliter les traductions ultérieures. L'accessibilité pour les publics en situation de handicap (notamment visuel) sera prise en compte à travers le respect des normes d'accessibilité (contrastes, navigation clavier, textes alternatifs).

2.6. Description graphique et ergonomique

2.6.1. Composants de la charte graphique

L'identité visuelle du site repose sur une charte moderne en rouge, noir et blanc.

Les éléments graphiques incluent :

- Un logo sobre et distinctif
- Une police lisible sans-serif, avec des tailles adaptées selon les supports
- Une palette principale : rouge vif, noir profond, blanc neutre
- Une cohérence graphique assurée sur toutes les pages

2.6.2. Design, Responsive design et autres exigences liées au design

Le site devra être **entièrement responsive** et offrir une **expérience utilisateur fluide** sur desktop, tablette et mobile. Le style visuel recherchée est un **design moderne**, légèrement minimaliste, avec des animations légères. La navigation sera claire, avec des appels à l'action visibles et une hiérarchie bien définie des contenus.

2.7. Besoins fonctionnels « métier »

2.7.1. Utilisateurs du projet

Les types d'utilisateurs concernés sont :

- Les visiteurs : navigation libre, inscription, achat de billets
- Les utilisateurs authentifiés : gestion de leur profil, historique d'achats
- Les organisateurs : création, édition et gestion d'événements
- Les administrateurs : modération, gestion des utilisateurs, statistiques

2.7.2. Informations relatives aux contenus

Les contenus disponibles incluent : fiches événements, visuels (affiches, photos), descriptions textuelles, vidéos intégrées, documents à télécharger (conditions générales, programmes). Le site respectera le RGPD : les données personnelles seront protégées, une politique de confidentialité sera affichée et le consentement explicite demandé pour la collecte de données.

2.7.3. Inventaire des besoins fonctionnels

Thème	Qui	Quoi	Pourquoi
Authentification	Utilisateur	Créer un compte	Participer à un événement
Authentification	Utilisateur	Se connecter / Se déconnecter	Gérer ses billets
Profil	Utilisateur	Modifier ses informations	Mettre à jour ses données
Gestion événements	Organisateur	Créer un événement	Le rendre visible au public
Billetterie	Visiteur	Acheter un billet	Participer à un événement
Administration	Admin	Modérer un événement	Maintenir la qualité du service
Statistiques	Admin	Consulter les indicateurs clés	Suivre les performances

Fonctionnalités supplémentaires envisagées :

- Espace client avec accès sécurisé
- Système de notification (email)
- Intégration d'un moteur de recherche interne
- Interface d'administration avec tableaux de bord
- Règles d'accès en fonction des rôles

2.8. Budget

Le budget total prévisionnel est estimé à **90 jours/homme** répartis comme suit :

Phase	Durée estimée
Analyse & Conception	15 j/h
Développement back-end	25 j/h
Développement front-end	20 j/h
Intégration & Tests	20 j/h
Documentation & Livraison	5 j/h
Réserves & imprévus	5 j/h

3. Présentation de l'entreprise et du service

Présentation de l'entreprise

M&C Billetterie est une jeune société française fondée en 2025, spécialisée dans les solutions de billetterie en ligne pour événements culturels, sportifs et professionnels de petite à moyenne envergure. Notre mission est de rendre la vente et le contrôle des billets simples, rapides et fiables pour les organisateurs comme pour le public. Basée en France et opérant en mode SaaS, M&C Billetterie s'adresse aux salles de spectacle, associations, festivals locaux, clubs sportifs, écoles et organisateurs d'événements B2B (salons, conférences). L'équipe, à taille humaine, réunit des profils techniques et opérationnels (produit, développement, support client, commercial) avec une culture de service et de transparence.

Présentation du service

La plateforme M&C Billetterie couvre l'ensemble du cycle de vie d'un événement: création (fiche événement, jauge, catégories de tarifs), mise en vente (page de vente responsive, paniers, codes promo), paiement sécurisé via un prestataire certifié, émission immédiate des billets (PDF/Wallet) et contrôle d'accès sur site (scan de QR codes via application mobile). Côté organisateur, un espace d'administration centralisé permet de suivre les ventes en temps réel, gérer les équipes (rôles simples), envoyer des confirmations et exporter les données essentielles (commandes, listes d'accès). Côté public, le parcours d'achat est pensé pour être clair et rapide: choix des billets, paiement, réception du billet et informations pratiques. La solution s'intègre avec les outils courants (emailing, paiement) et propose une API simple pour les besoins de base.

Positionnement et valeur apportée

M&C Billetterie mise sur la simplicité d'usage, la fiabilité opérationnelle les jours "chauds", et la proximité avec les organisateurs. Par rapport aux plateformes généralistes, notre proposition de valeur tient dans un outil léger, rapide à déployer, au coût maîtrisé et avec un accompagnement direct (support réactif, conseils de configuration). Pour les organisateurs, les bénéfices se mesurent en gain de temps,

réduction des erreurs de contrôle à l'entrée et meilleure visibilité sur les ventes. Pour le public, l'expérience est fluide et sécurisée.

Engagements et conformité

Les données sont hébergées en Europe et traitées conformément au RGPD (finalités claires, minimisation, droits des personnes). Les paiements sont délégués à un prestataire certifié (les données de carte ne transitent pas par nos serveurs). Nous suivons des indicateurs simples et concrets: disponibilité de la plateforme, temps de réponse du tunnel d'achat, taux d'échec de paiement, et taux de satisfaction des organisateurs.

Modèle économique

Le modèle est volontairement transparent: une commission par billet émis et, selon les besoins, un petit abonnement mensuel pour activer des fonctionnalités additionnelles (contrôle d'accès avancé, exports étendus, intégration API). Aucun frais caché, et des tarifs adaptés au volume des événements.

4. Gestion de projet

Le projet a été mené de manière itérative en m'appuyant sur des outils de gestion modernes. L'objectif était de garder une vision claire de l'avancement, de suivre les priorités (MVP, documentation, tests etc), et d'assurer la traçabilité du code et des décisions techniques.

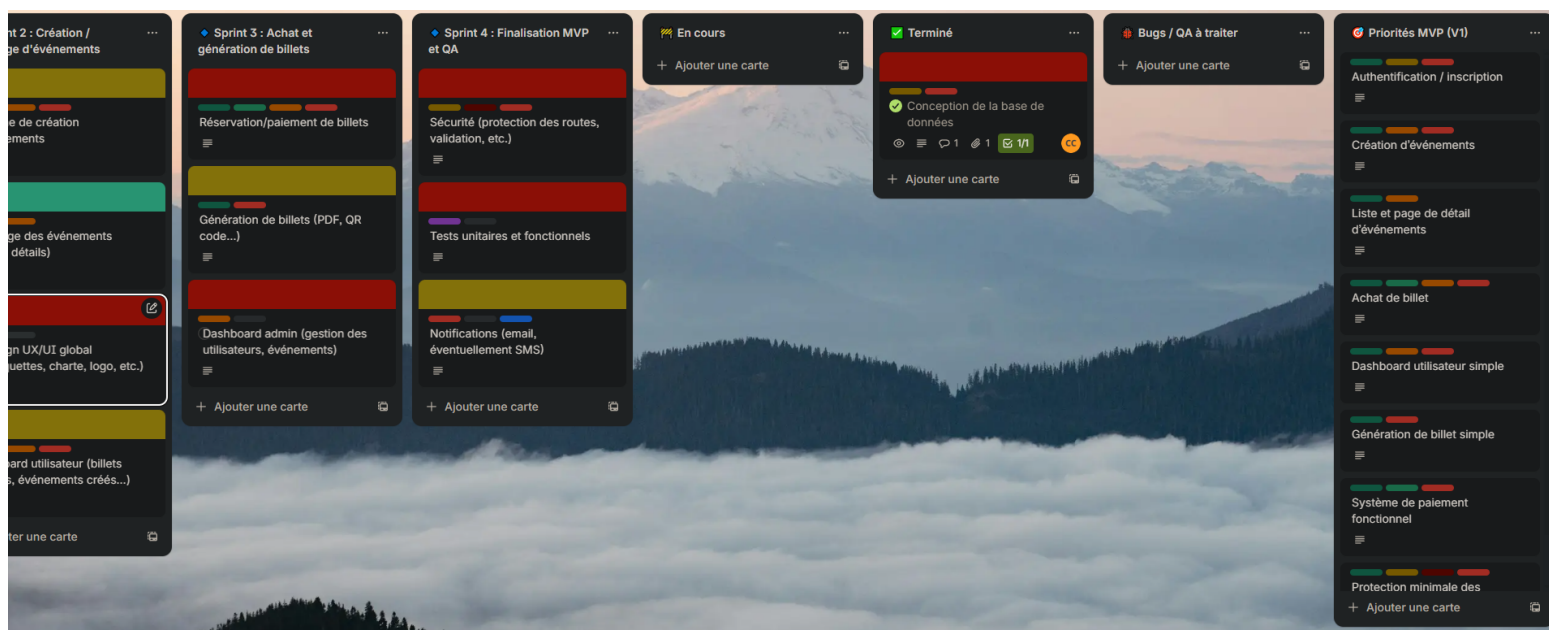
4.1 Méthodologie

Le projet a suivi une approche **itérative et incrémentale** : les fonctionnalités ont été développées par lots, testées, puis intégrées progressivement.

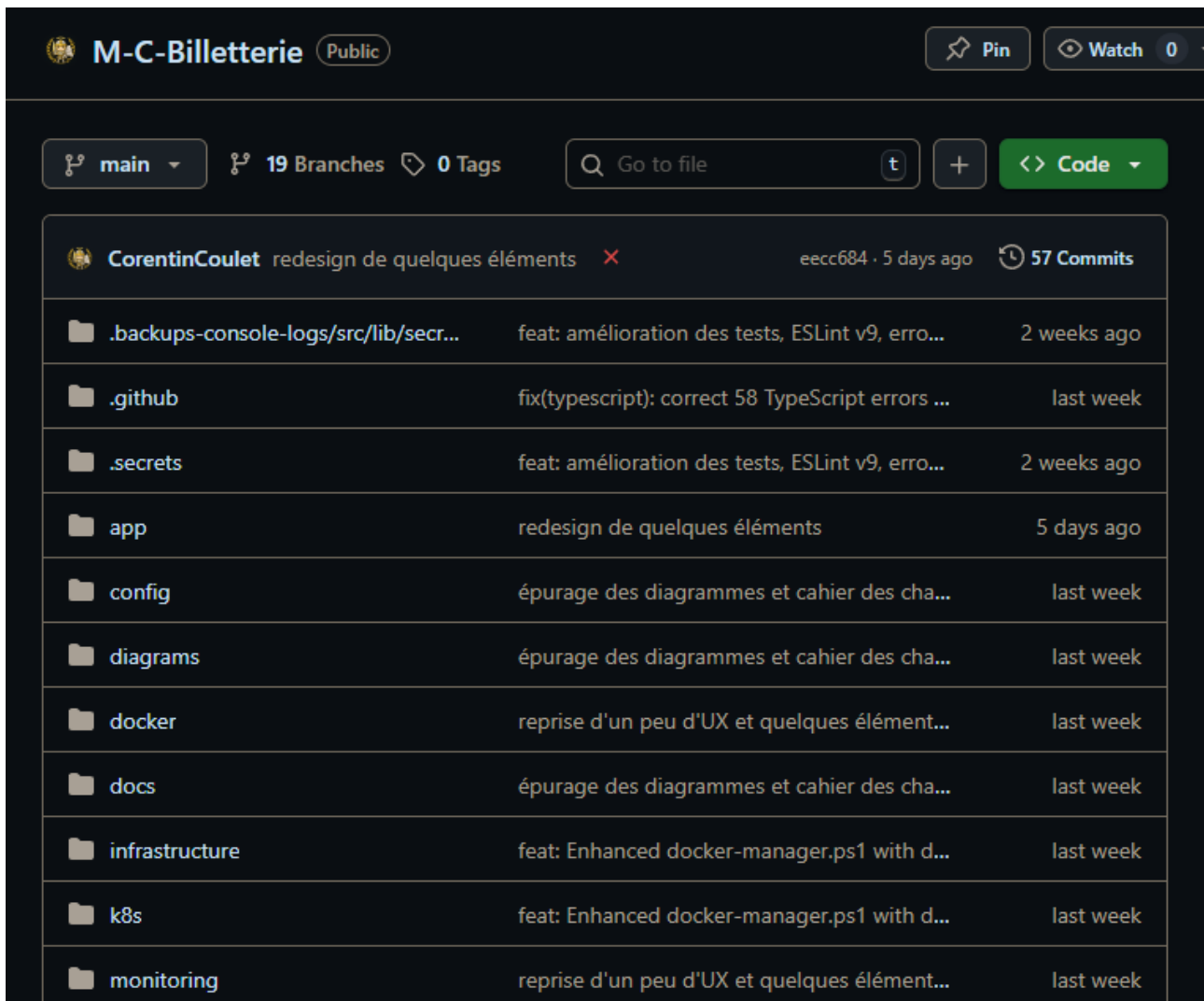
Cette méthode a permis d'ajuster certaines décisions en cours de route (organisation des DTOs, choix techniques, etc.).

4.2 Outils utilisés

- **Trello** : organisation des tâches en colonnes, permettant un suivi visuel et simple de l'avancement. (Backlog général, Sprint 1 Mise en place et fondations, Sprint 2 Création / Affichage d'évènements, Sprint 3 Achat et génération de billets, Sprint 4 Finalisation MVP et QA, En cours, Terminé, Bugs/QA à traiter, Priorités MVP V1)



- **GitHub** : gestion du code source, suivi des versions (Git), centralisation des commits et du travail effectué.



- **PlantUML (.puml)** : création des diagrammes (cas d'utilisation, activités, entités-relation, classes, architecture) afin d'assurer une modélisation claire et maintenable.

4.3 Suivi de l'avancement

- Les fonctionnalités majeures ont été planifiées selon les phases définies dans le cahier des charges (analyse, conception, développement, tests, livraison).
- Chaque tâche correspondait à une carte Trello, avec passage dans les colonnes au fur et à mesure.
- Les versions du code ont été balisées par des commits réguliers sur GitHub.

4.4 Gestion des risques et validations

- Une réserve de jours a été prévue pour gérer les imprévus (retards, corrections).
- Les diagrammes UML ont servi de référence pour valider les choix fonctionnels avant le codage.
- Les tests ont permis de confirmer la qualité des livrables avant la mise à disposition finale.

5. Spécifications fonctionnelles

L'application M&C Society repose sur une architecture en couches, avec une séparation claire entre les composants métier, la base de données, et les interfaces utilisateur. Les spécifications fonctionnelles suivantes ont été établies à partir du cahier des charges :

- Gestion des utilisateurs : inscription, connexion sécurisée, modification du profil.
- Gestion des rôles : visiteurs, organisateurs, administrateurs.
- Création, modification et suppression d'événements par les organisateurs.
- Achat de billets via une interface intuitive.
- Gestion d'un tableau de bord pour les administrateurs : statistiques, supervision des événements et utilisateurs.
- Interface responsive accessible depuis ordinateur, tablette et mobile.
- Respect des règles RGPD pour la gestion des données personnelles.

Diagrammes de cas d'utilisation :

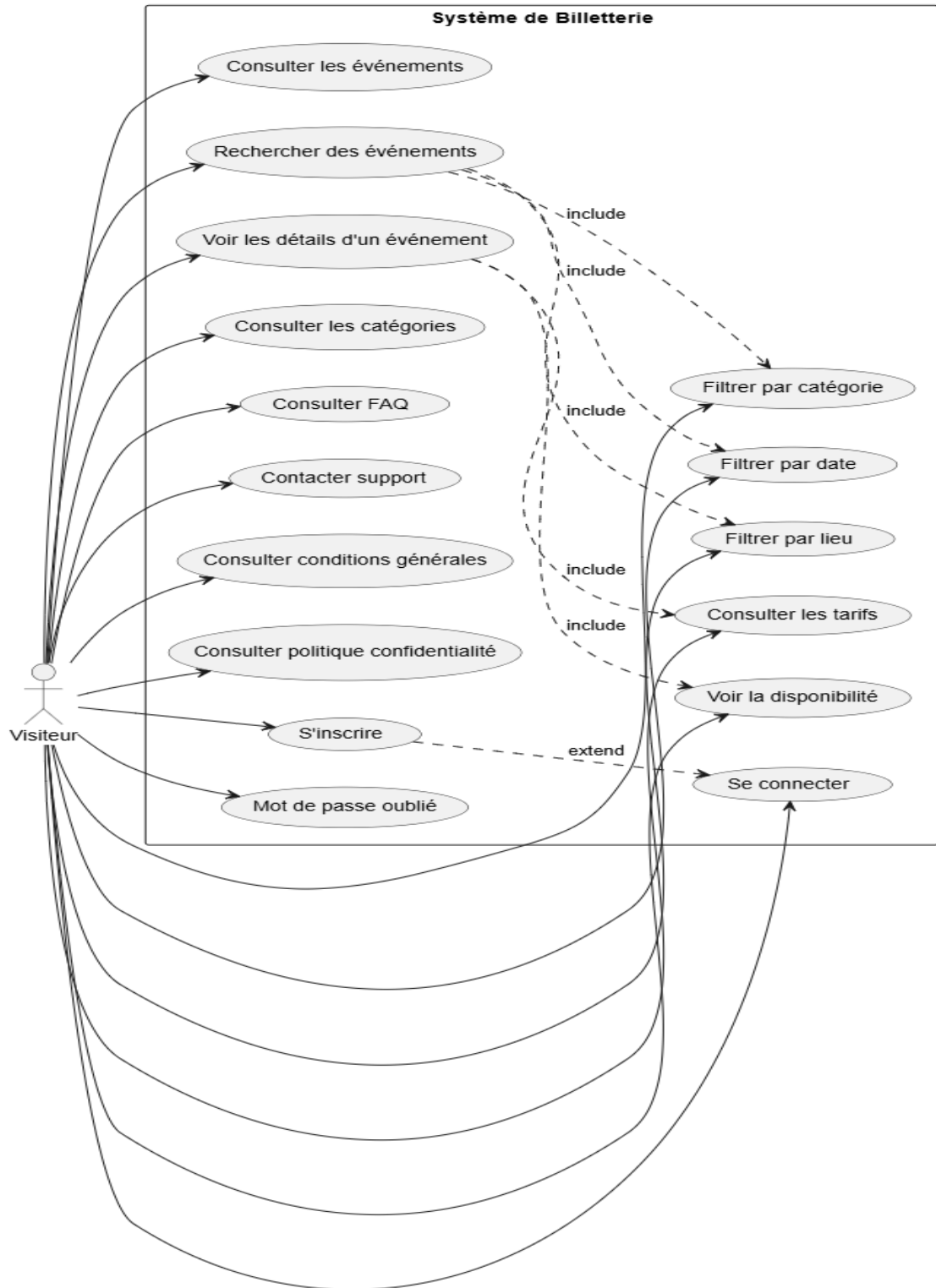
Rôle Visiteur :

Un visiteur non connecté peut explorer librement le catalogue d'événements disponibles sur la plateforme. Il consulte les événements, effectue des recherches et affine ses résultats en filtrant par catégorie, date ou lieu. Pour chaque événement, il accède aux détails complets incluant les tarifs et la disponibilité des billets.

Le visiteur navigue également dans les différentes catégories d'événements proposées et accède aux informations légales et pratiques du site : FAQ, conditions générales de vente et politique de confidentialité. En cas de besoin, il peut contacter le support client.

Pour aller plus loin et effectuer des achats, le visiteur doit créer un compte en s'inscrivant. Il peut également se connecter s'il possède déjà un compte, ou utiliser la fonction de récupération de mot de passe en cas d'oubli.

Diagramme de cas d'utilisation - Rôle Visiteur



Rôle Utilisateur :

Un utilisateur connecté hérite de toutes les fonctionnalités d'un visiteur et bénéficie d'un accès complet au système de billetterie. Il gère sa session en se déconnectant, réinitialisant son mot de passe ou vérifiant son adresse email.

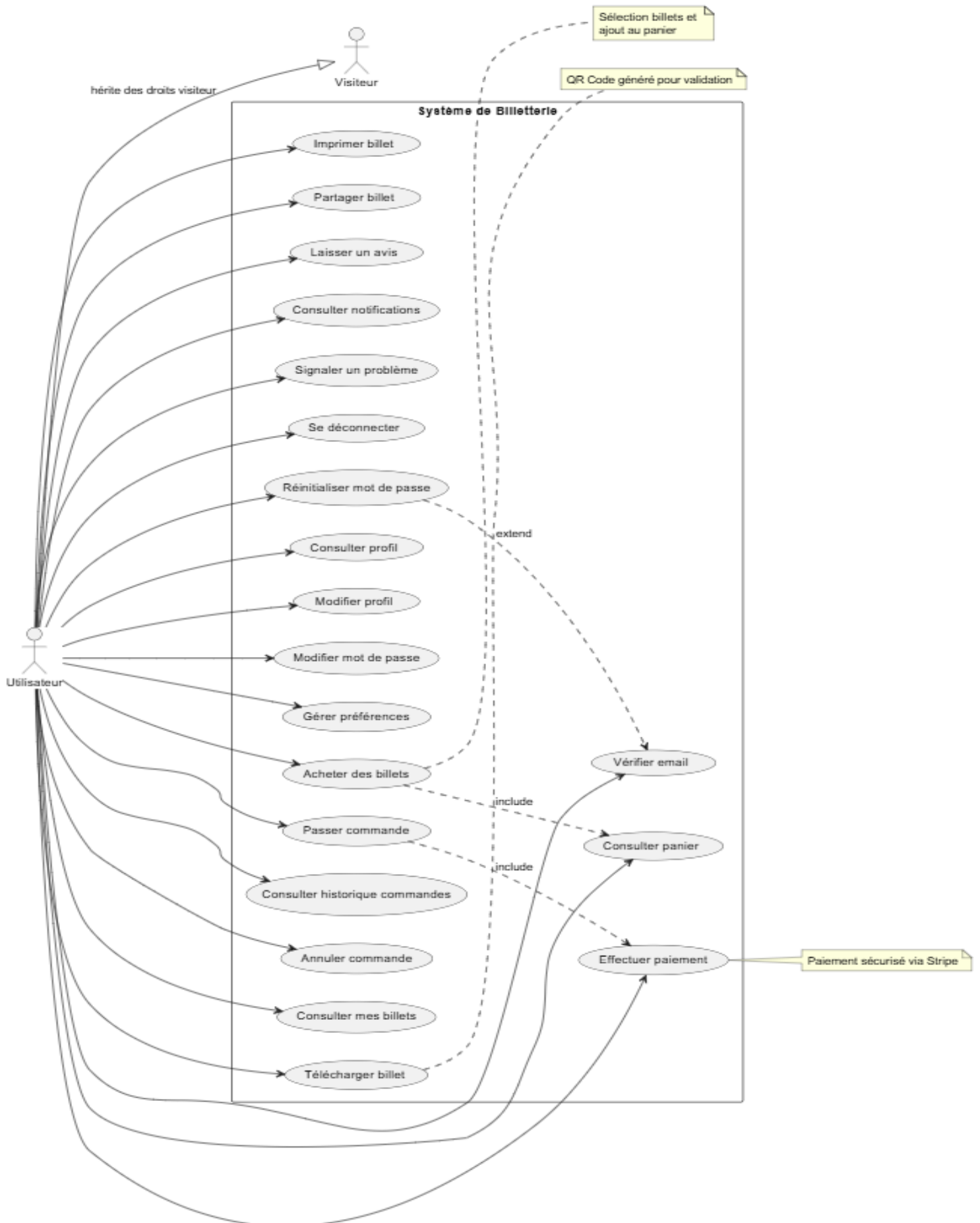
Son espace personnel lui permet de consulter et modifier son profil, changer son mot de passe et personnaliser ses préférences. Il accède à toutes ses notifications pour rester informé des actualités et mises à jour.

L'utilisateur peut acheter des billets en consultant son panier avant de passer commande. Le paiement s'effectue de manière sécurisée via Stripe. Il dispose d'un historique complet de ses commandes et peut les annuler si nécessaire avant l'événement.

Tous ses billets sont accessibles depuis son compte. Il peut les consulter, les télécharger au format électronique avec QR Code pour validation, les imprimer ou les partager avec d'autres personnes.

L'utilisateur interagit également avec la communauté en laissant des avis sur les événements auxquels il a participé. En cas de difficulté, il peut signaler un problème au support technique.

Diagramme de cas d'utilisation - Rôle Utilisateur



Rôle Organisateur :

Un organisateur hérite de tous les droits d'un utilisateur standard et dispose de fonctionnalités étendues pour gérer ses événements et leur commercialisation.

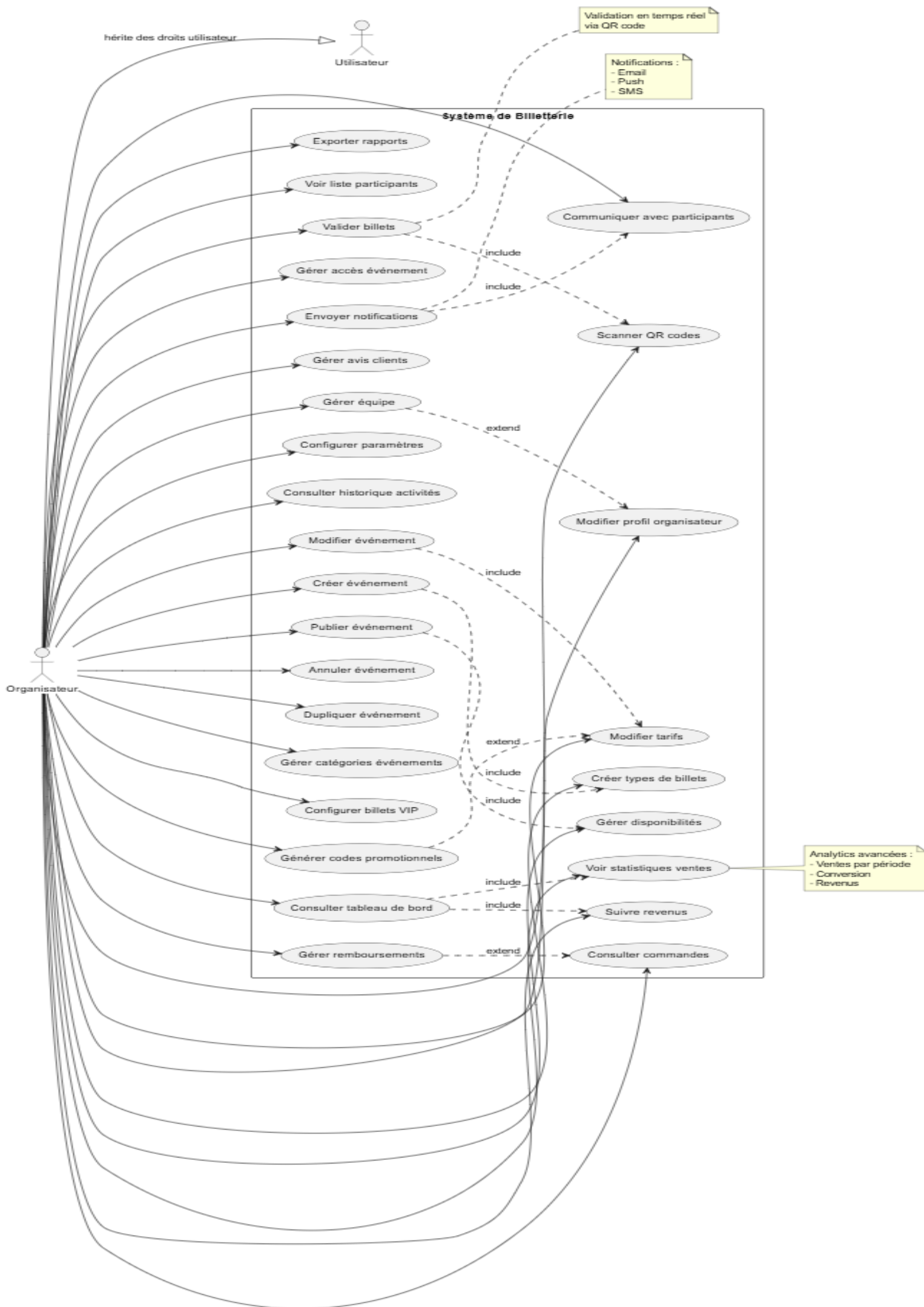
Il crée, modifie, publie ou annule ses événements, avec la possibilité de les dupliquer pour gagner du temps. Il gère également les catégories pour mieux organiser son catalogue. Pour chaque événement, il configure les différents types de billets, définit les tarifs, ajuste les disponibilités et peut créer des billets VIP. Il génère aussi des codes promotionnels pour stimuler les ventes.

Son tableau de bord lui offre une vue complète sur ses activités avec des statistiques détaillées : ventes par période, taux de conversion et suivi des revenus. Il consulte toutes les commandes, gère les remboursements si nécessaire et exporte des rapports pour ses analyses. Il accède à la liste complète des participants de ses événements et valide les billets en scannant les QR codes en temps réel lors de l'accès à l'événement.

L'organisateur communique directement avec ses participants en envoyant des notifications par email, push ou SMS. Il modère également les avis clients laissés sur ses événements.

Enfin, il personnalise son profil organisateur, gère son équipe de collaborateurs, configure les paramètres de son compte et consulte l'historique de toutes ses activités sur la plateforme.

Diagramme de cas d'utilisation - Rôle Organisateur



6. Spécifications techniques

Environnement technique :

- **Backend** : Next.js avec Prisma ORM (PostgreSQL)

Prisma est un ORM moderne pour JavaScript/TypeScript qui permet de manipuler une base de données avec du code plutôt que du SQL brut. Son principal avantage est la génération automatique d'un client entièrement typé à partir d'un schéma déclaratif, offrant une autocomplétion parfaite et détectant les erreurs avant l'exécution.

Comparé à d'autres ORM comme Sequelize ou TypeORM, Prisma se démarque par sa simplicité d'utilisation, son API intuitive et ses performances optimisées grâce à un moteur Rust. Il gère élégamment les migrations et s'adapte parfaitement aux architectures serverless, tout en fournissant une interface graphique (Prisma Studio) pour visualiser les données facilement.

- **Frontend** : Next.js

Next.js est un framework React qui facilite la création d'applications web modernes en ajoutant des fonctionnalités essentielles que React seul ne propose pas. Il permet notamment le rendu côté serveur (SSR) et la génération de sites statiques, améliorant considérablement les performances et le référencement SEO.

Ce qui distingue Next.js d'autres solutions comme Create React App, c'est son système de routing basé sur les fichiers, où chaque fichier dans le dossier pages ou `app` devient automatiquement une route. Il offre également l'optimisation automatique des images, le code splitting intelligent pour charger uniquement le JavaScript nécessaire, et supporte différentes stratégies de rendu (SSR, SSG, ISR) selon les besoins de chaque page.

Next.js intègre nativement TypeScript, propose des API routes pour créer des endpoints backend sans serveur séparé, et optimise automatiquement les performances en production. C'est devenu le choix privilégié pour les applications React professionnelles grâce à sa flexibilité et son écosystème mature soutenu par Vercel.

- **Base de données** : PostgreSQL (hébergé localement ou en ligne)

PostgreSQL est un système de gestion de base de données relationnelle open-source, robuste et mature. Il se distingue par sa conformité stricte aux standards SQL et son respect des normes ACID (Atomicité, Cohérence, Isolation, Durabilité), qui garantissent que toutes les transactions sont fiables et que les données restent intègres même en cas d'erreur ou de panne.

Comparé à MySQL ou SQLite, PostgreSQL offre des performances supérieures pour les requêtes sophistiquées et une richesse fonctionnelle impressionnante avec le support natif de JSON, des types de données avancés et des index spécialisés. Il excelle particulièrement dans la gestion de requêtes complexes et de charges de travail intensives, tout en restant gratuit et extensible. C'est devenu le choix standard pour les applications modernes nécessitant fiabilité et évolutivité, particulièrement adapté aux projets Node.js avec Prisma.

- **Authentication** : par mot de passe avec hachage (bcrypt)

Bcrypt est un algorithme de hachage de mots de passe conçu pour être intentionnellement lent, ce qui le rend résistant aux attaques par force brute. Il intègre automatiquement un "salt", c'est-à-dire une chaîne aléatoire unique ajoutée au mot de passe avant le hachage, qui garantit que deux utilisateurs avec le même mot de passe auront des hashes complètement différents. Il utilise également un "cost factor" paramétrable qui définit le nombre d'itérations de calcul. Plus ce facteur est élevé, plus le hachage prend du temps, permettant d'adapter la sécurité à l'évolution de la puissance des processeurs.

Pour valider un mot de passe lors de la connexion, on ne peut pas "retrouver" le mot de passe original car le hachage est irréversible. À la place, on hache le mot de passe saisi par l'utilisateur avec le même salt que celui stocké dans la base de données (qui est inclus dans le hash), puis on compare les deux hashes. S'ils sont identiques, le mot de passe est correct. Le hash final contient toutes les informations nécessaires (version, cost factor, salt et hash) pour effectuer cette vérification, ce qui en fait une solution autonome et sécurisée, largement recommandée par l'OWASP pour le stockage des mots de passe.

- **Outils** : Git, GitHub, Postman, Trello

Git est un système de contrôle de versions qui permet de suivre l'historique des modifications du code, de travailler en équipe sans écraser le travail des autres, et de revenir à des versions antérieures si nécessaire.

GitHub est une plateforme en ligne qui héberge des projets Git, facilitant la collaboration entre développeurs avec des fonctionnalités comme les pull requests, le suivi des issues et le partage de code open-source.

Postman est un outil pour tester et développer des API. Il permet d'envoyer des requêtes HTTP (GET, POST, PUT, DELETE) à vos endpoints, de visualiser les réponses et de documenter votre API de manière interactive.

Trello est un outil de gestion de projet basé sur des tableaux et des cartes. Il permet d'organiser les tâches visuellement, de suivre l'avancement du projet et de collaborer en équipe en déplaçant les cartes entre différentes colonnes (à faire, en cours, terminé).

Accessibilité :

- Contrastes respectés pour les malvoyants.
- Navigation possible au clavier.
- Texte alternatif sur les images.

Référencement :

- URLs propres.
- Titres et métadonnées dynamiques.
- Sitemap automatisé (à prévoir dans une version postérieure).

Diagramme de classes :

Le système de billetterie est structuré autour d'une architecture modulaire en sept blocs métier interconnectés, avec le Core Business comme élément central du dispositif.

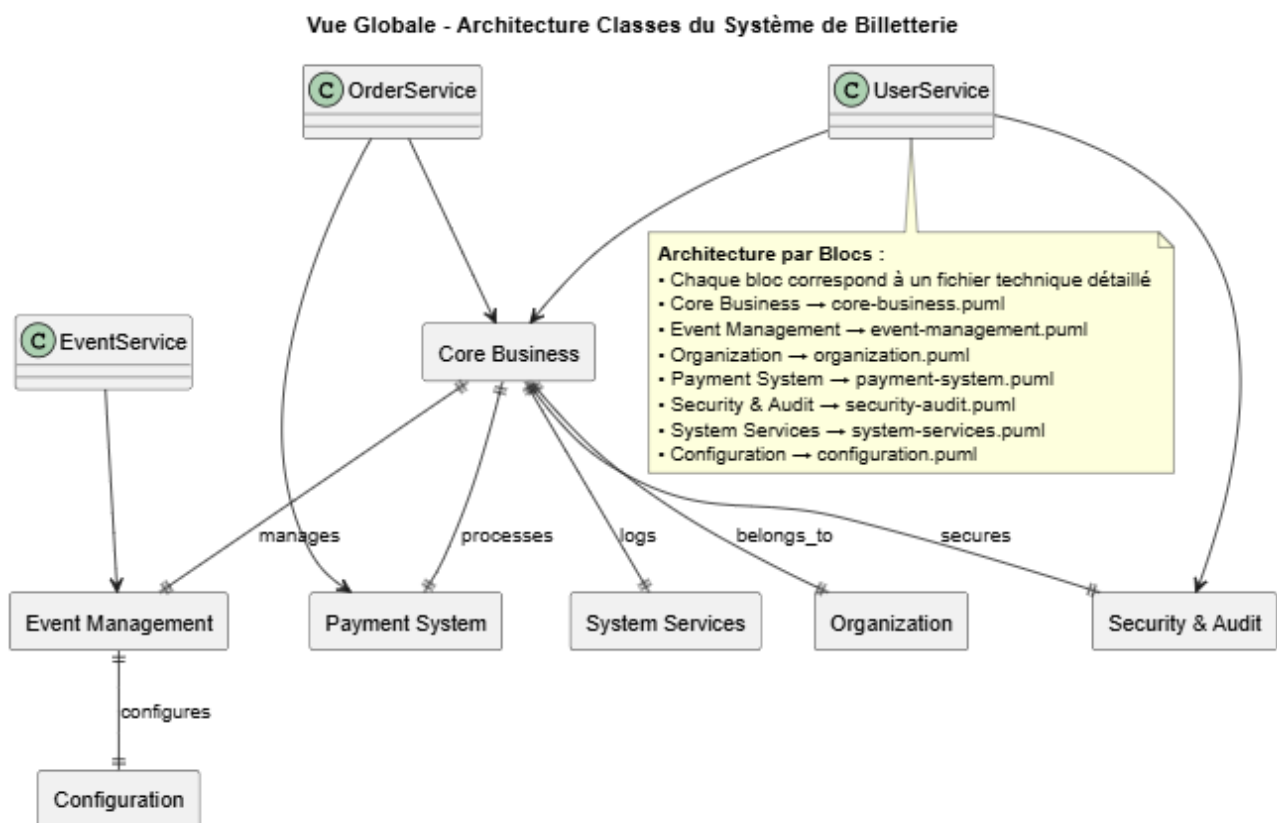
Le bloc Core Business constitue le noyau du système et maintient des relations bidirectionnelles avec l'ensemble des autres composants. Il communique directement avec l'Event Management pour la gestion des événements, s'appuie sur l'Organization pour structurer l'appartenance des entités, collabore avec le Payment System pour traiter les transactions financières, est sécurisé par le module Security & Audit, et utilise les System Services pour la journalisation et les notifications.

La configuration du système repose sur un lien entre l'Event Management et le bloc Configuration, permettant de paramétrer finement les événements proposés. Cette séparation assure une flexibilité dans l'adaptation du système aux besoins évolutifs.

L'orchestration des opérations métier est assurée par trois services clés qui agissent comme des chefs d'orchestre. L'EventService coordonne tout ce qui concerne la gestion événementielle, l'OrderService fait le pont entre le cœur métier et le système de paiement pour gérer l'intégralité du cycle de commande, tandis que l'UserService harmonise les interactions entre les utilisateurs, le Core Business et les mécanismes de sécurité et d'audit.

Cette conception favorise une séparation claire des responsabilités où chaque bloc technique correspond à un domaine fonctionnel spécifique, rendant le système à la fois maintenable et évolutif. Les fichiers de diagrammes détaillés permettent de plonger dans la structure interne de chaque module sans compromettre la lisibilité de la vue d'ensemble.

Diagramme global avec tous les blocs :



Bloc Organisation :

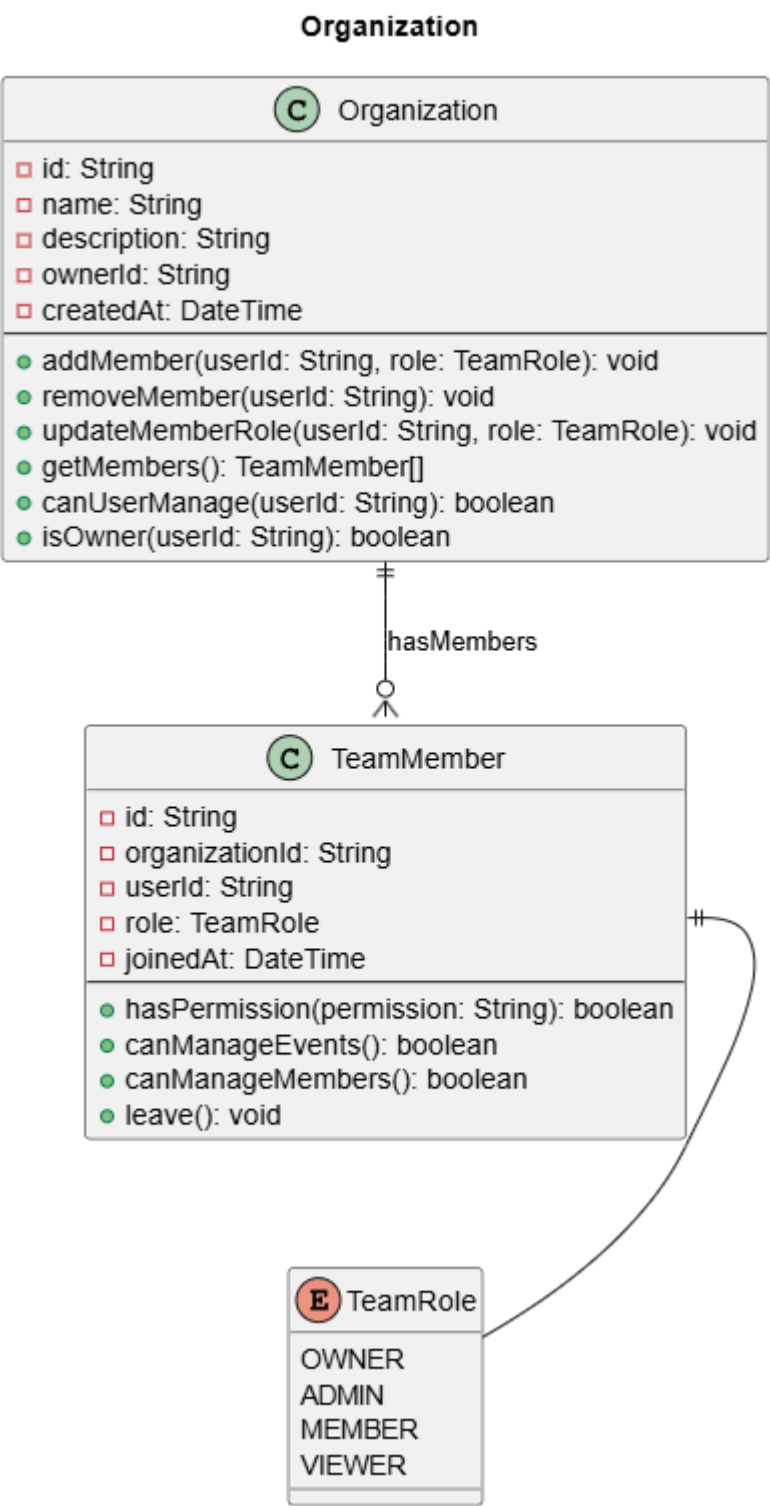


Diagramme d'architecture :

Le système de billetterie s'articule autour d'une architecture web moderne qui distingue trois types d'utilisateurs principaux interagissant avec la plateforme. Le client final recherche et achète des billets, l'organisateur crée et gère ses événements, tandis que l'administrateur supervise l'ensemble de la plateforme. Ces trois profils accèdent au système via une interface web unique en HTTPS.

Au cœur du dispositif se trouve une application web développée avec Next.js 15 qui offre une interface utilisateur responsive. Cette application communique avec une API REST également construite sur Next.js via des API Routes, assurant ainsi une cohérence technologique entre le front-end et le back-end. L'API constitue le cerveau du système en orchestrant l'ensemble de la logique métier et en exposant les endpoints nécessaires aux différentes opérations.

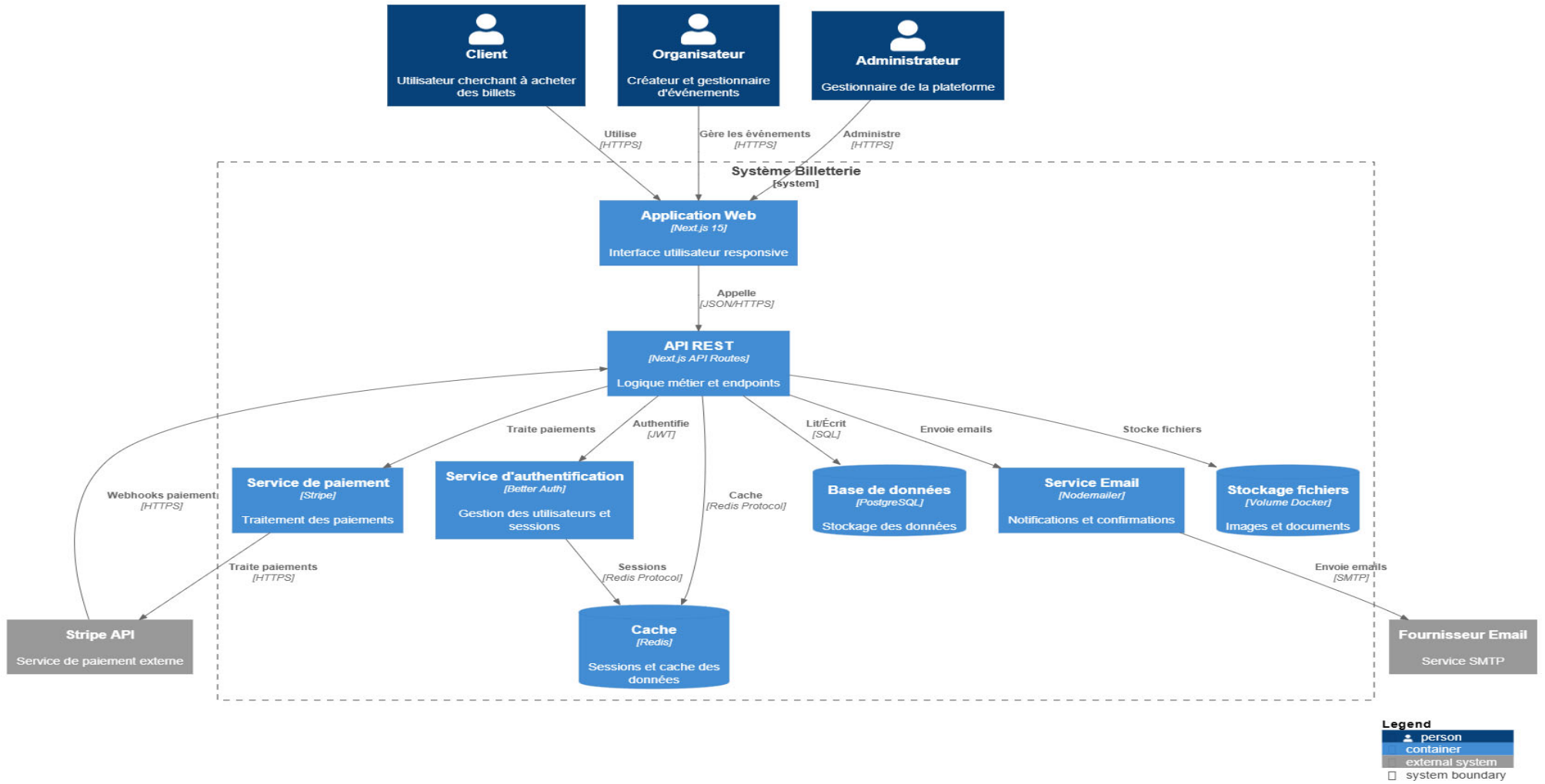
La sécurité et l'authentification sont prises en charge par Better Auth, un service dédié qui gère les utilisateurs, leurs sessions et leurs droits d'accès. Ce service s'appuie sur Redis pour maintenir les sessions actives de manière performante, permettant une expérience utilisateur fluide sans compromettre la sécurité.

Le système de stockage repose sur trois piliers complémentaires. PostgreSQL assure la persistance des données relationnelles avec robustesse, Redis optimise les performances en cachant les données fréquemment consultées et en maintenant les sessions utilisateurs, tandis qu'un volume Docker dédié héberge les fichiers statiques comme les images d'événements et les documents générés.

Les services externes jouent un rôle crucial dans l'écosystème. Stripe gère l'intégralité du cycle de paiement via son API et communique les confirmations de transaction au système par webhooks, garantissant la synchronisation en temps réel des états de commande. Le service email, implémenté avec Nodemailer, se connecte à un fournisseur SMTP pour délivrer les notifications, confirmations de commande et billets électroniques aux utilisateurs.

Cette architecture assure une séparation claire des responsabilités où chaque conteneur remplit une fonction spécifique, facilitant la maintenance, la scalabilité et l'évolution du système. La génération de QR codes est intégrée directement dans l'API comme module interne, évitant la complexité d'un service séparé tout en maintenant la flexibilité nécessaire.

Vue d'ensemble du système - Billetterie

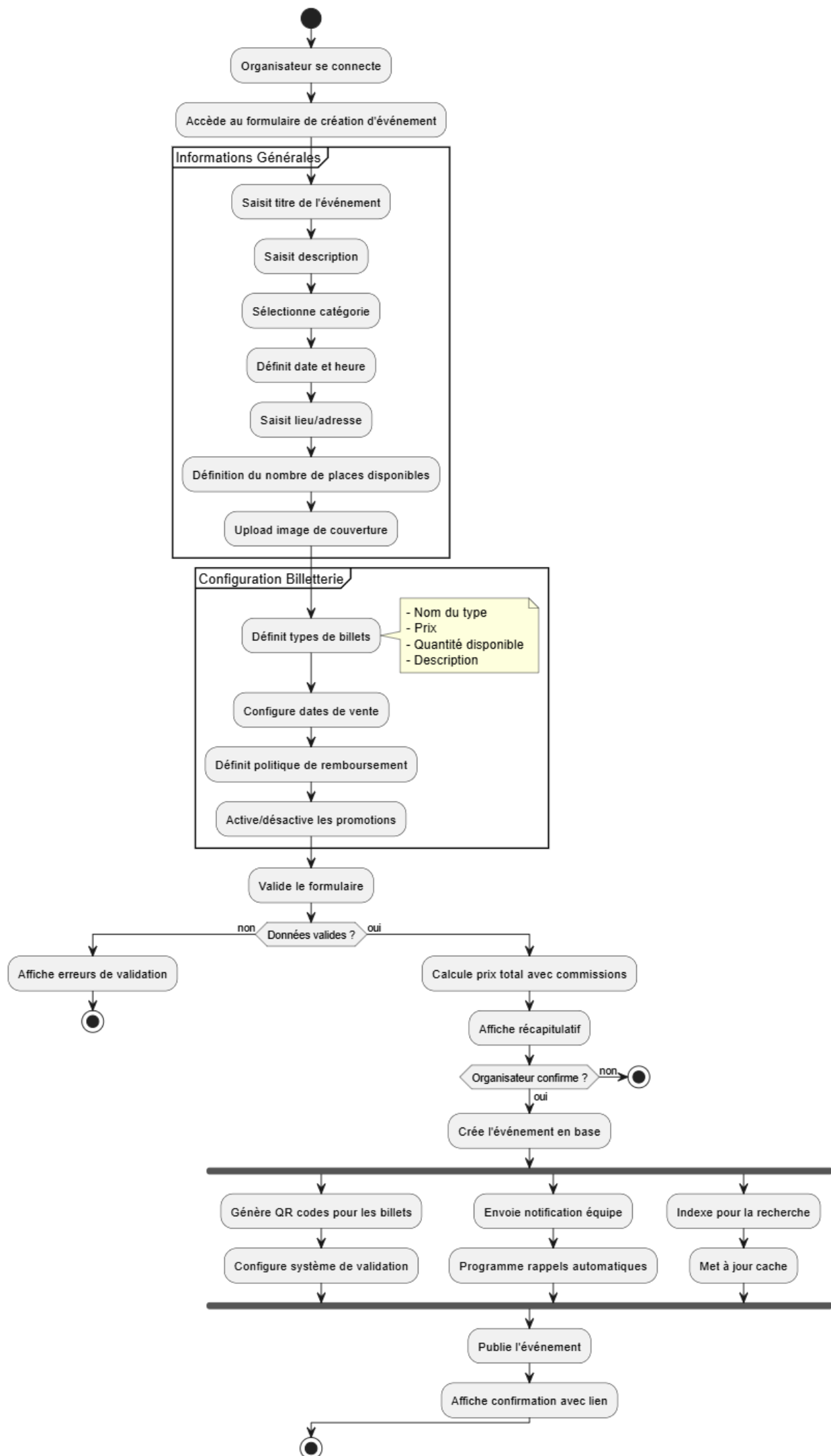


Diagrammes d'activités :

La création d'événement commence par la connexion de l'organisateur qui accède ensuite au formulaire dédié. Il y saisit d'abord les informations générales comprenant le titre, la description, la catégorie, la date, l'heure, le lieu, le nombre de places et peut ajouter une image de couverture. Il configure ensuite la billetterie en définissant les différents types de billets avec leurs prix et quantités, les dates de vente, la politique de remboursement et les options de promotion.

Après validation du formulaire, si des erreurs sont détectées, elles sont affichées et le processus s'arrête. Si tout est correct, le système calcule le prix total avec les commissions et présente un récapitulatif. Une fois confirmé par l'organisateur, l'événement est créé en base de données. Le système lance alors plusieurs tâches en parallèle : génération des QR codes et configuration de la validation, envoi des notifications et programmation des rappels, ainsi que l'indexation pour la recherche et la mise à jour du cache. L'événement est finalement publié et un message de confirmation avec le lien est affiché à l'organisateur.

Création d'Événement



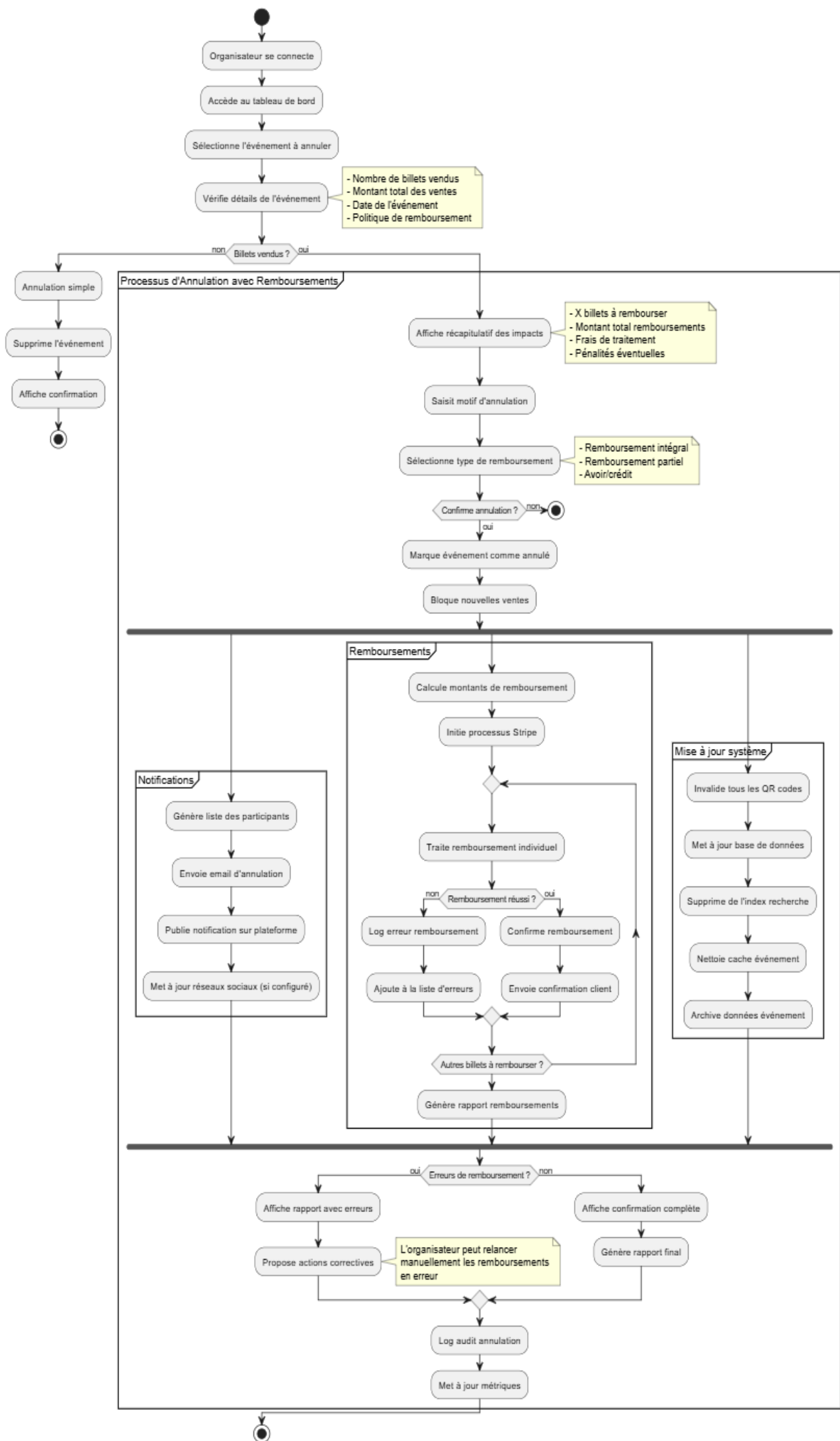
L'annulation d'événement commence lorsque l'organisateur se connecte, accède à son tableau de bord et sélectionne l'événement concerné. Le système vérifie les détails incluant le nombre de billets vendus, le montant des ventes et la politique de remboursement.

Si aucun billet n'a été vendu, l'événement est simplement supprimé. Si des billets ont été vendus, l'organisateur visualise un récapitulatif des impacts, saisit un motif d'annulation et choisit le type de remboursement souhaité. Après confirmation, l'événement est marqué comme annulé et les nouvelles ventes sont bloquées.

Le système lance alors trois processus parallèles : l'envoi des notifications aux participants par email et sur la plateforme, le traitement des remboursements via Stripe avec journalisation des erreurs éventuelles, et la mise à jour du système qui invalide les QR codes, actualise la base de données, supprime l'événement de la recherche et archive les données.

En cas d'erreurs de remboursement, un rapport avec actions correctives est affiché permettant de relancer manuellement les transactions échouées. Sinon, une confirmation complète avec rapport final est présentée. L'opération est enregistrée dans les logs d'audit et les métriques sont mises à jour.

Annulation d'Événement par l'Organisateur



L'annulation de billet commence lorsque l'utilisateur se connecte, accède à ses billets et sélectionne celui à annuler. Le système vérifie les détails incluant l'événement, la date, le prix et la politique de remboursement.

Si le délai d'annulation est dépassé, un message indique l'impossibilité d'annuler et propose de contacter l'organisateur. Si l'événement est annulé ou reporté, des options de remboursement automatique ou d'avoir sont proposées. Pour un événement normal, le système vérifie l'éligibilité au remboursement, calcule le montant selon la politique définie et affiche un récapitulatif. L'utilisateur doit confirmer pour poursuivre.

Une fois confirmé, le billet est marqué comme annulé, le QR code est invalidé et la place est libérée. Le système lance quatre processus parallèles : le remboursement via Stripe avec gestion des erreurs éventuelles, l'envoi d'un email de confirmation avec mise à jour du statut de commande, la notification de l'organisateur avec actualisation des statistiques, et l'enregistrement dans les logs d'audit avec mise à jour des métriques.

Si une liste d'attente existe, le premier inscrit est notifié et peut acheter le billet libéré. L'utilisateur reçoit la confirmation d'annulation avec le délai de remboursement, peut donner un feedback et voit son interface mise à jour.

Annulation de Billet par l'Utilisateur



Diagramme d'entité-relation :

Vue d'Ensemble

Le système de billetterie est conçu autour d'une architecture modulaire où chaque module assume des responsabilités spécifiques tout en communiquant efficacement avec les autres composants. Au cœur de cette architecture se trouve le concept d'utilisateur multi-rôles, qui permet à une même entité User de porter différentes casquettes selon les besoins : simple utilisateur, organisateur d'événements, ou administrateur système.

Le Module Core : Le Pilier Central

Le module Core constitue la fondation du système en gérant les entités fondamentales que sont les utilisateurs (User), les commandes (Order) et les billets (Ticket). C'est ici que s'établit la gestion de base des utilisateurs, incluant leurs profils et leurs rôles. Ce module agit comme un hub central puisque presque tous les autres modules interagissent avec lui d'une manière ou d'une autre, que ce soit pour créer des commandes, attribuer des rôles ou gérer l'authentification des utilisateurs.

Le Module Events : La Vitrine du Système

Le module Events prend en charge tout ce qui concerne la gestion des événements, depuis leur création jusqu'à leur publication. Il englobe les événements eux-mêmes (Event), leurs catégories (Category) et les lieux où ils se déroulent (Venue). Ce module travaille en étroite collaboration avec le module Core pour générer les tickets associés aux événements, et s'appuie sur le module Configuration pour personnaliser l'apparence et recueillir les avis des participants.

Le Module Organizer : La Dimension Professionnelle

Le module Organizer ajoute une couche d'organisation professionnelle au système en permettant la création de structures organisationnelles (Organization) et la gestion d'équipes (TeamMember). Ce qui est particulièrement intéressant ici, c'est que les membres d'équipe ne sont autres que des utilisateurs du système ayant endossé le rôle d'organisateur. Ces organisations sont ensuite responsables de la création et de la gestion des événements qu'elles proposent au public.

Le Module Payment : Le Cœur Financier

Le module Payment gère l'intégralité des transactions financières du système, du traitement des paiements à l'émission des factures. Il est directement lié au module Core puisque chaque paiement correspond à une commande passée par un utilisateur. Ce module assure la traçabilité complète des transactions et garantit l'intégrité financière de la plateforme.

Le Module Configuration : La Personnalisation

Le module Configuration offre les outils nécessaires pour personnaliser l'expérience utilisateur à travers les thèmes visuels (Theme), la gestion des avis et commentaires (Reviews), ainsi que l'internationalisation via les traductions. Il intervient principalement au niveau des événements pour permettre aux organisateurs de personnaliser la présentation de leurs manifestations et aux utilisateurs de partager leurs retours d'expérience.

Le Module Security : Le Gardien du Système

Le module Security assure la protection de l'ensemble de la plateforme en gérant les sessions utilisateurs (UserSession), en traçant toutes les actions sensibles (AuditLog), en surveillant les tentatives de connexion suspectes (LoginAttempt) et en bloquant les adresses IP malveillantes (BlockedIP). Ce module travaille main dans la main avec le module Core pour sécuriser les sessions des utilisateurs et alimente le module System en informations de sécurité pour une surveillance globale.

Le Module System : L'Orchestrateur Technique

Le module System prend en charge les aspects techniques et opérationnels de la plateforme, notamment la journalisation des activités (ActivityLog), l'envoi de notifications aux utilisateurs, la gestion des sauvegardes système (SystemBackup) et le monitoring général. Il collecte les informations en provenance de tous les autres modules, en particulier celles relatives à la sécurité, et assure que les utilisateurs restent informés des événements qui les concernent.

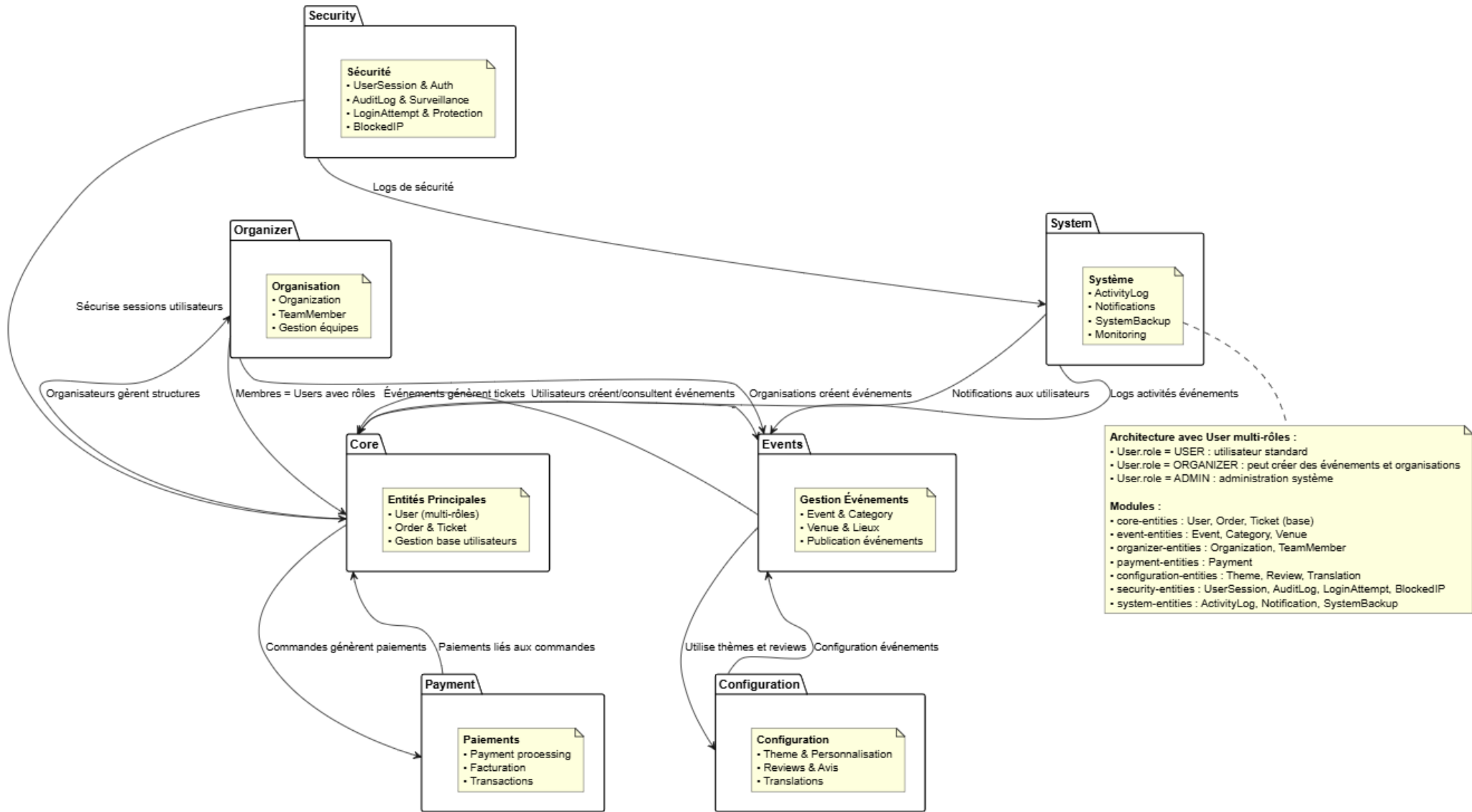
Les Interactions Inter-Modules

L'architecture se distingue par la richesse de ses interactions. Les utilisateurs du module Core créent et consultent les événements du module Events, génèrent des commandes qui déclenchent des paiements dans le module Payment, et peuvent devenir organisateurs en créant des structures dans le module Organizer. Les

organisations ainsi créées prennent ensuite la main sur la création d'événements, qui eux-mêmes génèrent des tickets pour les utilisateurs. Le module Security veille sur toutes ces opérations en sécurisant les sessions et en journalisant les actions critiques, pendant que le module System orchestre les notifications et maintient l'ensemble du système opérationnel.

Cette architecture modulaire permet une séparation claire des responsabilités tout en maintenant une cohérence globale, facilitant ainsi la maintenance, l'évolution et la scalabilité de la plateforme de billetterie.

Vue Globale - Système de Billetterie



Merise :

Modèle Conceptuel de Données (MCD) :

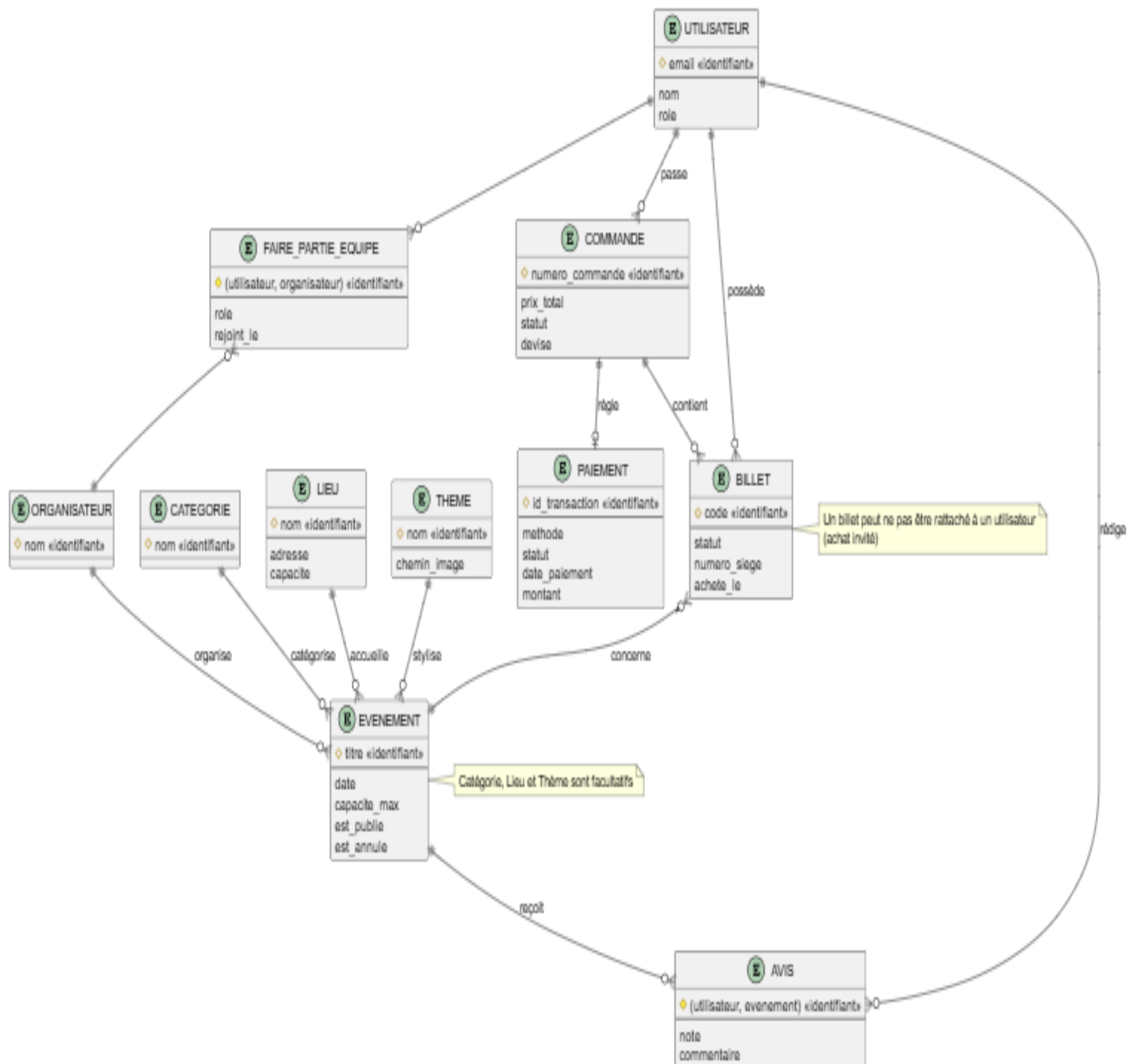
Au cœur du système se trouvent les UTILISATEUR et les EVENEMENT, qui constituent les deux piliers principaux. Les utilisateurs, identifiés par leur email, nom et rôle, peuvent interagir avec le système de plusieurs manières. Ils passent des commandes pour acquérir des billets, rédigent des avis sur les événements auxquels ils participent, et possèdent directement les billets qu'ils ont achetés. Toutefois, le système prévoit également la possibilité d'achats invités, où un billet peut exister sans être rattaché à un utilisateur spécifique.

Les EVENEMENT sont décrits par leur titre, date et capacité maximale, avec des indicateurs permettant de savoir s'ils sont publiés ou annulés. Chaque événement est organisé par un ORGANISATEUR, entité distincte possédant son propre nom. Cette séparation permet à un même organisateur de gérer plusieurs événements. Pour enrichir la description des événements, trois attributs optionnels sont disponibles : une CATEGORIE pour classifier l'événement, un LIEU définissant l'endroit où se déroule la manifestation avec son adresse et sa capacité, et un THEME qui permet d'appliquer une stylisation visuelle personnalisée.

Le processus d'achat est modélisé à travers les COMMANDE, qui agrègent un ensemble de BILLET pour un montant total dans une devise spécifique. Chaque commande possède un statut permettant de suivre son cycle de vie. Les billets générés contiennent un code unique, un statut, un numéro de siège et une date d'achat, et sont rattachés à un événement particulier. Le règlement financier s'effectue via un PAIEMENT, qui enregistre la méthode utilisée, le statut du paiement, sa date et son montant.

Une relation particulière mérite attention : l'association FAIRE_PARTIE_EQUIPE qui lie les utilisateurs aux organisateurs. Cette relation porteuse d'attributs permet de modéliser l'appartenance d'un utilisateur à l'équipe d'un organisateur, en précisant son rôle et la date à laquelle il a rejoint cette équipe. Cette structure autorise des configurations flexibles où un utilisateur peut collaborer avec plusieurs organisateurs.

Enfin, les AVIS créent un lien de retour entre les utilisateurs et les événements. Un utilisateur rédige des avis comprenant une note et un commentaire, tandis que les événements reçoivent ces évaluations, permettant ainsi de constituer une base de feedback pour améliorer l'expérience utilisateur et la qualité des événements proposés.



Modèle Logique de Données (MLD) :

Le Modèle Logique de Données transforme la vision conceptuelle en une structure technique prête à être implémentée dans une base de données relationnelle, en ajoutant tous les détails nécessaires à son fonctionnement opérationnel.

Chaque entité est maintenant dotée d'un identifiant unique sous forme d'UUID servant de clé primaire, garantissant l'unicité et la traçabilité de chaque enregistrement. Les dates de création et de modification sont systématiquement ajoutées pour assurer un audit temporel complet des données. L'entité UTILISATEUR s'enrichit considérablement avec un mot de passe hashé, un système de vérification d'email avec horodatage, un suivi de la dernière connexion et de la dernière modification du mot de passe. Un champ métadonnées au format JSON permet de stocker des informations additionnelles sans rigidifier le schéma, offrant ainsi une grande flexibilité évolutive.

Les EVENEMENT intègrent désormais plusieurs clés étrangères pointant vers leurs entités associées : l'organisateur qui le gère, la catégorie optionnelle qui le classe, le lieu qui l'accueille et le thème qui définit son apparence visuelle. Deux nouveaux attributs font leur apparition : l'autorisation d'achat anonyme qui permet de vendre des billets sans compte utilisateur, et l'autorisation de transfert qui détermine si un billet peut être cédé à une autre personne. La description passe d'un simple texte à un champ de type Text permettant des contenus plus étoffés.

La gestion des BILLET devient particulièrement sophistiquée avec l'ajout d'un système de QR code dynamique. Chaque billet possède un QR code actuel qui peut être régénéré selon un intervalle de rotation configurable, renforçant ainsi la sécurité contre la fraude et la duplication. Le système trace précisément quand le QR code a été généré, si le billet a été scanné et utilisé, avec les horodatages correspondants. La relation avec l'utilisateur peut être nulle pour gérer les achats anonymes, tandis que les clés étrangères vers l'événement et la commande restent obligatoires pour maintenir l'intégrité référentielle.

Les COMMANDE intègrent un système de promotion avec un code promo et un montant de remise associé, permettant de gérer des stratégies commerciales variées. Le champ métadonnées peut contenir des informations complémentaires comme l'adresse IP de l'acheteur ou des détails sur le parcours d'achat. La devise est explicitement stockée pour gérer des transactions internationales.

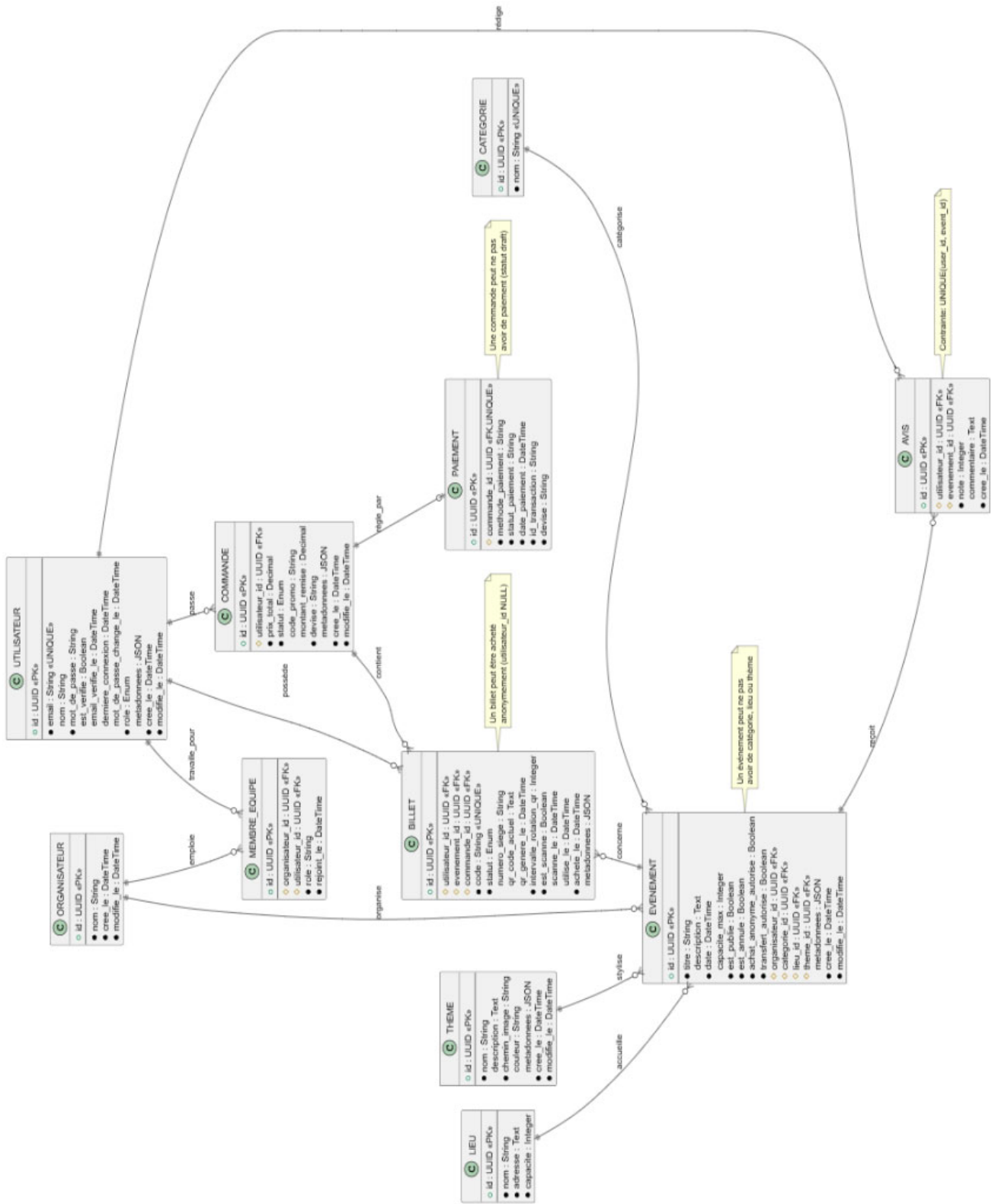
L'entité PAIEMENT établit une relation un-à-un avec la commande via une clé étrangère unique, garantissant qu'une commande ne peut avoir qu'un seul paiement associé. Elle conserve l'identifiant de transaction fourni par le processeur de paiement, la méthode utilisée et le statut du paiement, créant ainsi une piste d'audit complète. Une commande peut exister sans paiement si elle est en statut brouillon, permettant de gérer le panier avant validation.

La relation MEMBRE_EQUIPE matérialise l'association porteuse d'attributs du MCD en une table de liaison complète avec son propre identifiant. Elle relie utilisateurs et organisateurs tout en capturant le rôle spécifique de chaque membre et sa date d'entrée dans l'équipe. Cette structure permet des requêtes efficaces pour retrouver tous les membres d'un organisateur ou tous les organisateurs auxquels un utilisateur contribue.

Les entités CATEGORIE, LIEU et THEME deviennent des tables de référence complètes. Le lieu détaille maintenant précisément son adresse et sa capacité physique. Le thème s'enrichit d'une couleur principale et de métadonnées JSON pour stocker des paramètres de stylisation supplémentaires. La catégorie maintient une contrainte d'unicité sur son nom pour éviter les doublons.

L'entité AVIS impose une contrainte d'unicité composite sur le couple utilisateur-événement, empêchant ainsi un utilisateur de laisser plusieurs avis pour le même événement. La note est typée comme entier, suggérant probablement un système d'évaluation sur une échelle fixe.

Ce modèle logique traduit donc la conception abstraite en un schéma de base de données robuste, avec des contraintes d'intégrité, des types de données précis, et des mécanismes de sécurité et d'audit intégrés, tout en maintenant la flexibilité nécessaire pour l'évolution future du système grâce aux champs métadonnées JSON.



Modèle Physique de Données (MPD) :

Le Modèle Physique de Données représente l'implémentation concrète dans PostgreSQL, spécifiant tous les détails techniques nécessaires au déploiement effectif de la base de données, incluant les types de données précis, les contraintes, les index de performance et les mécanismes de sécurité.

La table `users` adopte des types PostgreSQL spécifiques avec des `VARCHAR` limitées à 255 caractères pour l'email et le nom, garantissant une taille raisonnable tout en conservant la flexibilité. Le mot de passe est stocké dans un `VARCHAR(255)` dimensionné pour accueillir les hash `bcrypt` ou `argon2`. Les champs temporels utilisent systématiquement `TIMESTAMPTZ` pour gérer les fuseaux horaires, évitant ainsi les problèmes de décalage horaire dans un contexte international. Un type `ENUM` personnalisé nommé `ROLE` définit les rôles possibles avec une valeur par défaut `'USER'`. Plusieurs index sont créés sur l'email et le rôle pour accélérer les requêtes de connexion et de filtrage par profil. Un trigger optionnel peut automatiser la mise à jour du champ `updated_at` à chaque modification d'enregistrement.

Les events implémentent une indexation composite sophistiquée combinant date et statut de publication, optimisant les requêtes fréquentes qui cherchent les événements publiés dans une période donnée. Le champ `location` reste en `VARCHAR(255)` malgré l'existence de la relation avec `venues`, probablement pour gérer des cas où l'événement n'a pas de lieu fixe ou pour des informations complémentaires comme une salle spécifique. Des contraintes de vérification peuvent être ajoutées via SQL pour s'assurer que la date est future et qu'un événement annulé ne peut être publié. Le champ `metadata` en `JSONB` permet des requêtes performantes sur le contenu JSON contrairement au type `JSON` standard.

La table `orders` définit le prix avec `DECIMAL(10,2)` offrant une précision exacte essentielle pour les transactions financières, évitant les erreurs d'arrondi des nombres flottants. Une contrainte vérifie que le prix total est positif ou nul, et une autre peut contrôler que le montant de remise ne dépasse pas le prix total. Le type `ENUM` `ORDER_STATUS` encode les différents états possibles : `draft` pour un panier non validé, `pending_payment` en attente de règlement, `paid` pour les commandes payées, et `cancelled` pour les annulations. Un index composite sur `user_id` et `status` optimise les requêtes listant les commandes d'un utilisateur par statut.

Les tickets présentent une gestion avancée avec plusieurs horodatages traçant le cycle de vie complet : `purchased_at` pour l'achat, `qr_code_generated_at` pour la génération

du QR code, `scanned_at` pour le scan à l'entrée, et `used_at` pour la validation finale. L'intervalle de rotation du QR code est stocké en entier représentant probablement des heures, avec une valeur par défaut de 12 heures. Un index partiel sur les tickets actifs permet d'optimiser les requêtes ne ciblant que les billets valides en excluant ceux utilisés ou annulés. Un trigger peut générer automatiquement le code unique du billet lors de l'insertion.

La table `payments` établit une contrainte `UNIQUE` sur `order_id` matérialisant physiquement la relation un-à-un avec les commandes. Cette contrainte empêche au niveau base de données qu'une commande puisse avoir plusieurs paiements, garantissant l'intégrité même en cas d'erreur applicative. L'identifiant de transaction conserve la référence fournie par le processeur de paiement externe comme Stripe ou PayPal.

Les venues imposent une contrainte de capacité strictement positive, empêchant la création de lieux avec une capacité nulle ou négative. Cette validation au niveau base de données constitue un dernier rempart contre les données incohérentes.

Les thèmes incluent une validation potentielle du format couleur via une expression régulière PostgreSQL vérifiant que la valeur correspond au format hexadécimal standard `#RRGGBB`. Le chemin d'image utilise un `VARCHAR(500)` permettant de stocker des URL complètes ou des chemins relatifs assez longs.

La table `reviews` implémente une contrainte `UNIQUE` composite sur le couple `user_id` et `event_id`, traduisant physiquement la règle métier qui empêche un utilisateur de laisser plusieurs avis sur le même événement. Une contrainte peut valider que la note est comprise entre 1 et 5, cohérente avec un système d'évaluation par étoiles standard.

La table `team_members` utilise également une contrainte `UNIQUE` composite garantissant qu'un utilisateur ne peut apparaître qu'une seule fois dans l'équipe d'un organisateur donné, bien qu'il puisse faire partie de plusieurs équipes différentes.

Le modèle intègre un système de sécurité et d'audit exhaustif avec `user_sessions` qui trace chaque session utilisateur avec son token, adresse IP, user agent et informations sur l'appareil. Le suivi de `last_activity_at` permet d'implémenter des mécanismes de timeout, tandis que `expires_at` gère l'expiration automatique des sessions. Une session peut être détruite avec une raison enregistrée dans `destroy_reason`.

Les `activity_logs` et `event_logs` fournissent deux niveaux de traçabilité différents, le premier pour les actions utilisateur sur des ressources spécifiques identifiées par leur

type et ID, le second pour des événements système plus généraux avec un contexte textuel.

La table `login_attempts` capture toutes les tentatives de connexion réussies ou échouées avec leur adresse IP et user agent, permettant de détecter des patterns d'attaque par force brute. Un champ `risk_score` peut stocker une évaluation du risque calculée par l'application, tandis que `session_id` lie éventuellement la tentative à une session créée.

Les `audit_logs` constituent un journal d'audit complet avec un hash d'événement garantissant l'intégrité et la non-répudiation des actions. Le niveau de risque et le marqueur de sensibilité permettent de classifier les événements pour une analyse ultérieure ou des obligations de conformité réglementaire.

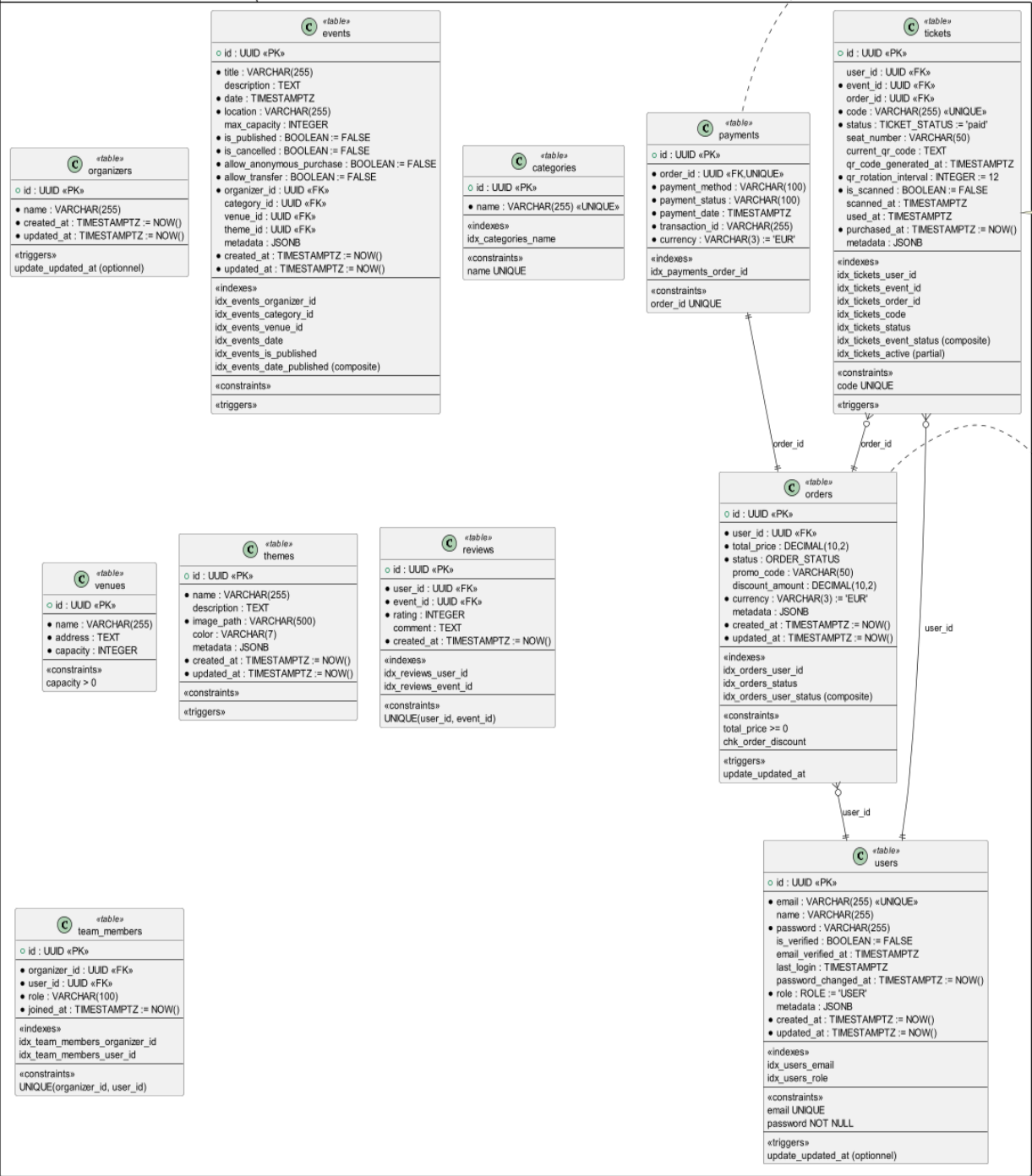
La table `blocked_ip` gère le blocage d'adresses IP suspectes avec possibilité d'expiration automatique, un compteur de blocages successifs, et la traçabilité de qui a débloqué l'IP et quand.

Les `system_backups` enregistrent toutes les sauvegardes système avec leur type, statut, chemin de fichier chiffré, clé de chiffrement, taille et checksum SHA-256 pour vérifier l'intégrité. La durée de rétention configurable permet une gestion automatisée du cycle de vie des sauvegardes.

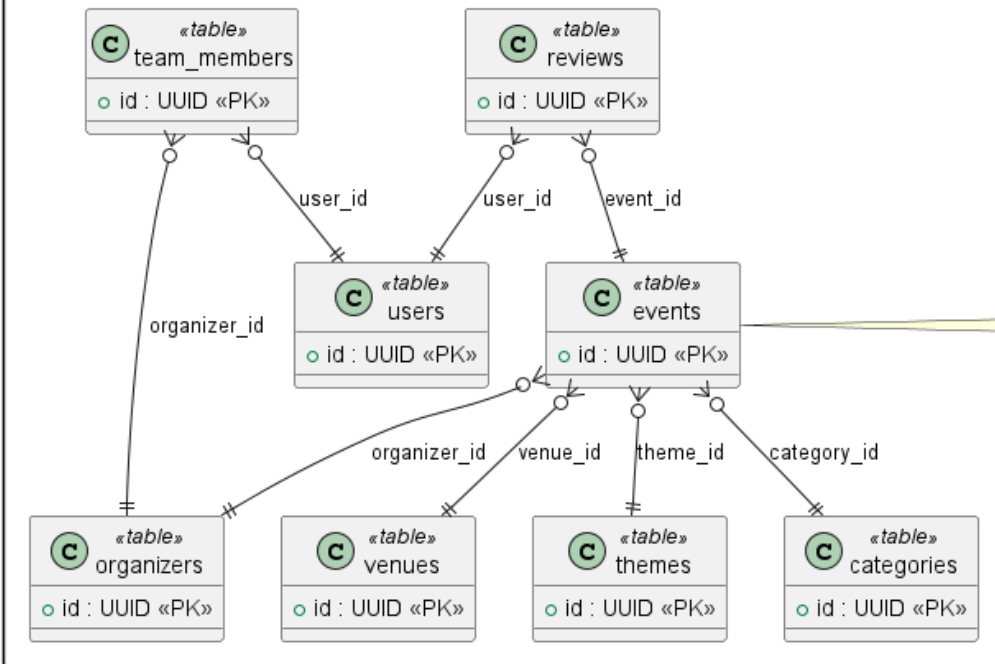
La `password_history` conserve l'historique des mots de passe hashés d'un utilisateur, permettant d'implémenter une politique empêchant la réutilisation de mots de passe récents, renforçant ainsi la sécurité.

Ce modèle physique traduit donc non seulement la structure logique en instructions SQL concrètes avec types de données PostgreSQL spécifiques, mais y intègre également une couche complète de sécurité, d'audit et de performance via indexes stratégiques, contraintes d'intégrité robustes et tables de traçabilité exhaustives, créant ainsi un système prêt pour un environnement de production exigeant.

Base de Données PostgreSQL - Système de Billetterie



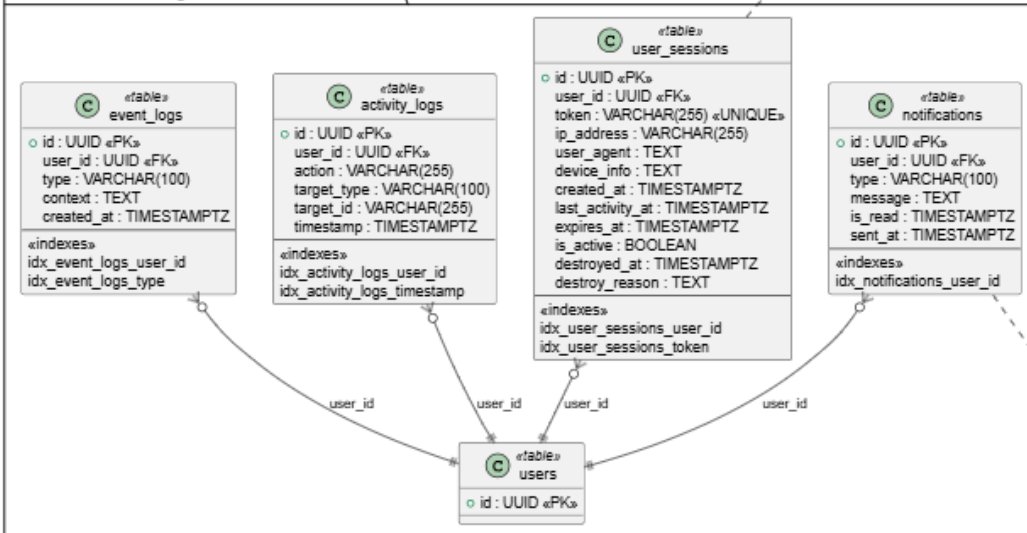
Base de Données PostgreSQL - Événements & Organisateur



Contraintes métier :

- Date future
- Pas publié si annulé
- Capacité > 0

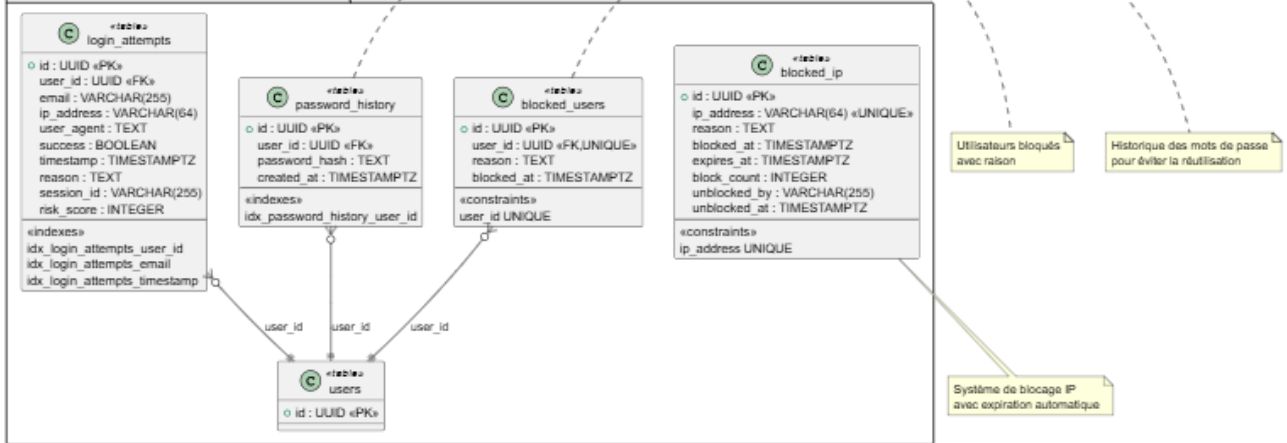
Base de Données PostgreSQL - Sessions & Activités



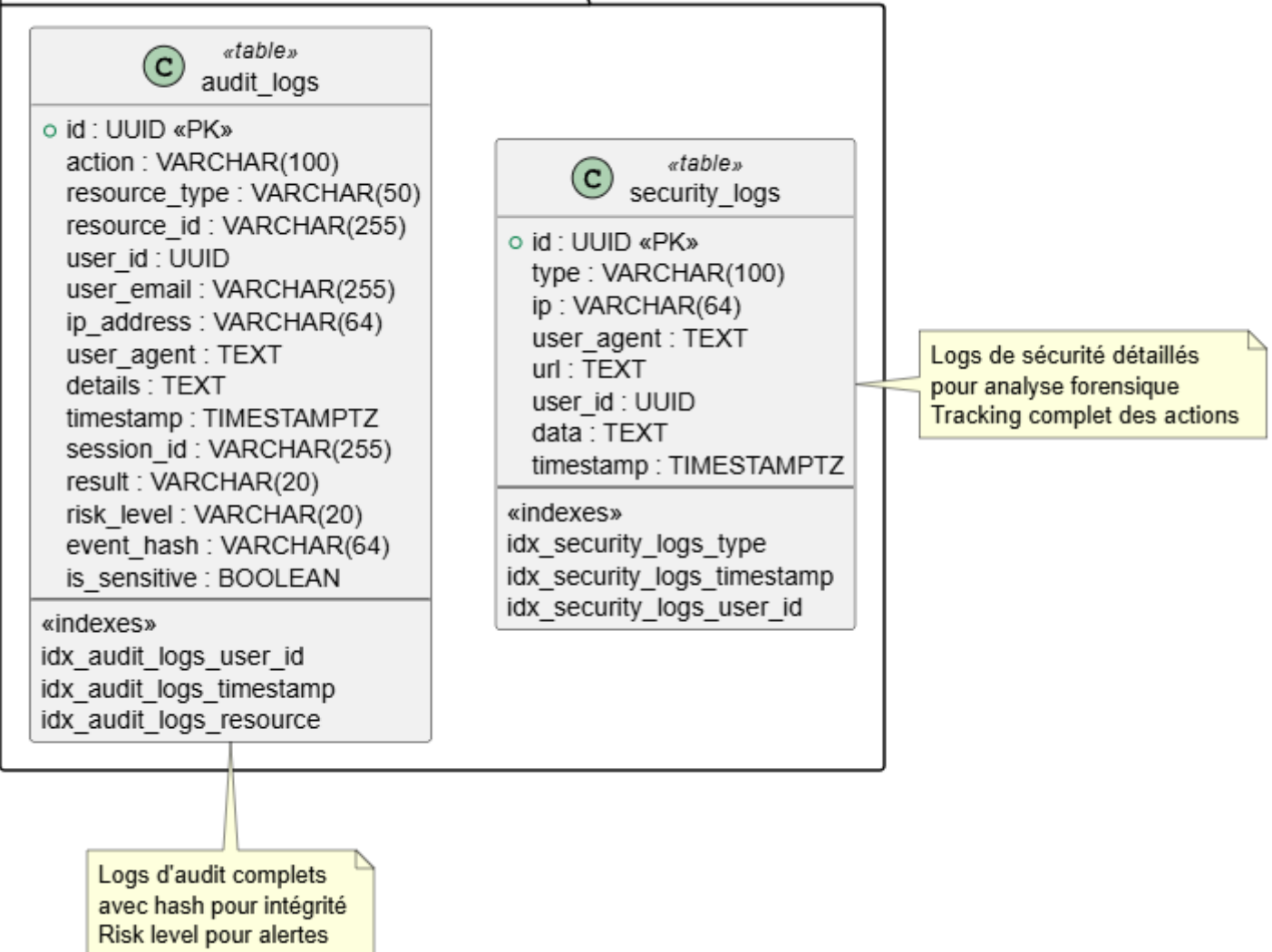
Gestion des sessions utilisateur
 avec tracking IP et device
 Expiration automatique

Système de notifications
 avec suivi de lecture

Base de Données PostgreSQL - Sécurité Utilisateurs



Base de Données PostgreSQL - Logs & Audit



Base de Données PostgreSQL - Système & i18n

«table» translations
- id : UUID «PK» - entity_type : VARCHAR(100) - entity_id : VARCHAR(255) - locale : VARCHAR(10) - field : VARCHAR(100) - value : TEXT
«indexes» idx_translations_entity idx_translations_locale
«constraints» UNIQUE(entity_type, entity_id, locale, field)

Système i18n
Traductions par entité
et par locale

«table» system_backups
- id : UUID «PK» - type : VARCHAR(50) - status : VARCHAR(50) - file_path : TEXT - encryption_key : TEXT - file_size : BIGINT - checksum_sha256 : VARCHAR(64) - started_at : TIMESTAMPTZ - completed_at : TIMESTAMPTZ - error : TEXT - retention_days : INTEGER
«indexes» idx_system_backups_status idx_system_backups_started_at

Gestion des sauvegardes
avec chiffrement et checksum
Rétention configurable

7. Réalisations

Fonctionnalités clés développées :

- **Inscription et connexion sécurisée**

[← Retour à l'accueil](#)

Authentification

Connexion

Inscription

 Particulier

 Entreprise

Informations personnelles

Prénom *

Nom *

Email *

coulet.corentin@gmail.com

Téléphone

+33 6 12 34 56 78

Date de naissance

jj/mm/aaaa



Sécurité

Mot de passe *

.....

Minimum 8 caractères

Confirmer le mot de passe *

☐ J'accepte les [conditions d'utilisation](#) et la [politique de confidentialité](#)

S'inscrire comme particulier

The image shows a web form for authentication. At the top left, there is a link '← Retour à l'accueil'. The main title is 'Authentification'. Below it are two tabs: 'Connexion' (active) and 'Inscription'. There are two input fields: 'Email' with the value 'coulet.corentin@gmail.com' and 'Mot de passe' with masked characters '*****'. At the bottom is a blue button labeled 'Se connecter'.

- **Système de création et de gestion d'événements**
- **Achat de billets avec vérification des places disponibles**
- **Filtrage d'événements par catégorie, date ou lieu**
- **Tableau de bord administrateur avec statistiques clés**

Exemple de code intégré :

- DTO et validation lors de l'inscription.

Cette validation côté client offre un feedback immédiat à l'utilisateur sans attendre la réponse du serveur. Elle améliore l'expérience utilisateur (UX) en évitant des allers-retours réseau inutiles. Cependant, elle ne remplace jamais la validation serveur qui reste obligatoire pour la sécurité.



```
1  const handleRegister = async (e: React.FormEvent) => {
2    e.preventDefault()
3    setLoading(true)
4
5    if (registerData.password !== registerData.confirmPassword) {
6      alert('Les mots de passe ne correspondent pas')
7      setLoading(false)
8      return
9    }
10
11    if (!registerData.acceptTerms) {
12      alert('Vous devez accepter les conditions d\'utilisation')
13      setLoading(false)
14      return
15    }
16
17    if (userType === 'organizer' && (!registerData.companyName || !registerData.siret)) {
18      alert('Veuillez remplir les informations de votre entreprise')
19      setLoading(false)
20      return
21    }
22
23    // Appel API...
24  }
```

Zod permet de définir un schéma déclaratif qui valide ET type les données simultanément. La méthode `.refine()` ajoute des règles métier personnalisées. Toute donnée non conforme est rejetée avant même d'atteindre la base de données, évitant ainsi les injections et données corrompues.



```

1  import { z } from 'zod';
2
3  const registerSchema = z.object({
4    email: z.string().email('Email invalide'),
5    password: z.string().min(8, 'Le mot de passe doit contenir au moins 8 caractères'),
6    name: z.string().min(2, 'Le nom doit contenir au moins 2 caractères').optional(),
7    confirmPassword: z.string().optional(),
8  }).refine((data) => {
9    if (data.confirmPassword && data.password !== data.confirmPassword) {
10      return false;
11    }
12    return true;
13  }, {
14    message: "Les mots de passe ne correspondent pas",
15    path: ["confirmPassword"],
16  });
17
18  async function handleRegister(request: NextRequest) {
19    const { data, error } = await validateBody(request, registerSchema);
20    if (error) return error;
21
22    const result = await authService.register(data.email, data.password, data.name);
23  }

```

Les cookies httpOnly ne sont pas accessibles par JavaScript côté client, ce qui empêche les attaques XSS (Cross-Site Scripting) de voler le token. L'attribut sameSite: 'strict' protège contre les attaques CSRF (Cross-Site Request Forgery). En production, secure: true force HTTPS.



```

1  if (result.token) {
2    response.cookies.set('auth-token', result.token, {
3      httpOnly: true,
4      secure: process.env.NODE_ENV === 'production',
5      sameSite: 'strict',
6      maxAge: 60 * 60 * 24 * 7,
7    });
8  }

```

Le service implémente la logique métier : vérification d'unicité, hash du mot de passe avec bcrypt (algorithme à coût adaptatif), et création sécurisée. Le select Prisma garantit que le mot de passe n'est jamais retourné dans la réponse. Cette couche constitue la défense finale contre les données invalides.

```
1  async register(email: string, password: string, name?: string): Promise<LoginResult | null> {
2    try {
3      if (!email || !password) {
4        throw new Error('Email and password are required');
5      }
6
7      if (password.length < 8) {
8        throw new Error('Password must be at least 8 characters long');
9      }
10
11     const existingUser = await prisma.user.findUnique({
12       where: { email }
13     });
14     if (existingUser) {
15       throw new Error('User already exists');
16     }
17
18     const hashedPassword = await bcrypt.hash(password, 10);
19
20     const user = await prisma.user.create({
21       data: {
22         email,
23         password: hashedPassword,
24         name,
25         role: 'USER'
26       },
27       select: {
28         id: true,
29         email: true,
30         name: true,
31         role: true,
32       }
33     });
34
35     return { user, token };
36   } catch (error) {
37     console.error('Registration error:', error);
38     return null;
39   }
40 }
```


Les DTOs (Data Transfer Objects) définissent des contrats explicites entre les couches. RegisterDto pour l'entrée, UserResponseDto pour la sortie. TypeScript garantit qu'aucun développeur ne peut accidentellement exposer le mot de passe hashé. C'est du type safety appliqué à la sécurité.

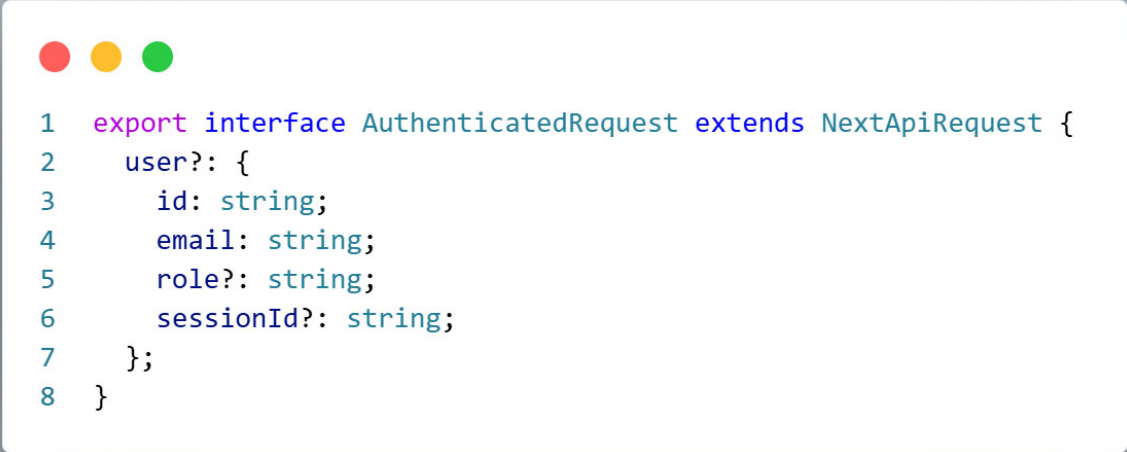


```
1  export interface RegisterDto {
2    email: string;
3    password: string;
4    name?: string;
5  }
6
7  export interface UserResponseDto {
8    id: string;
9    email: string;
10   name: string;
11   role: UserRole;
12   isVerified: boolean;
13   createdAt: Date;
14 }
```

- Middleware d'authentification.

Ce middleware assure la sécurité des routes API en vérifiant l'authentification des utilisateurs via des tokens JWT et en validant leurs sessions en base de données. Il

propose également un système de contrôle d'accès basé sur les rôles (RBAC) pour restreindre l'accès à certaines fonctionnalités selon les privilèges utilisateur.



```
1  export interface AuthenticatedRequest extends NextApiRequest {  
2    user?: {  
3      id: string;  
4      email: string;  
5      role?: string;  
6      sessionId?: string;  
7    };  
8  }
```



```
1 export async function isAuthenticated(  
2   req: NextApiRequest,  
3   res: NextApiResponse,  
4   next: NextHandler  
5 ) {  
6   try {  
7     const token = req.cookies.token || req.headers.authorization?.replace('Bearer ', '');  
8  
9     if (!token) {  
10      return res.status(401).json({ message: 'Authentication required' });  
11    }  
12  
13    const user = await authService.validateToken(token);  
14  
15    if (!user || !user.sessionId) {  
16      return res.status(401).json({ message: 'Invalid or expired token' });  
17    }  
18  
19    const session = await prisma.userSession.findUnique({ where: { id: user.sessionId } });  
20    if (!session || session.expiresAt < new Date()) {  
21      return res.status(401).json({ message: 'Session expirée' });  
22    }  
23  
24    (req as AuthenticatedRequest).user = user;  
25  
26    return next();  
27  } catch (error) {  
28    console.error('Authentication error:', error);  
29    return res.status(401).json({ message: 'Authentication failed' });  
30  }  
31 }
```



```
1 export function hasRoles(roles: string[]) {  
2   return async (req: NextApiRequest, res: NextApiResponse, next: NextHandler) => {  
3     try {  
4       await isAuthenticated(req, res, async () => {  
5         const user = (req as AuthenticatedRequest).user;  
6         if (!user || !user.role || !roles.includes(user.role)) {  
7           return res.status(403).json({ message: 'Insufficient permissions' });  
8         }  
9         return next();  
10      });  
11    } catch (error) {  
12      console.error('Authorization error:', error);  
13      return res.status(403).json({ message: 'Insufficient permissions' });  
14    }  
15  };  
16 }
```

- Requête Prisma pour créer un événement lié à un utilisateur.

Exemple avec la création d'une commande de billets : Cette fonction crée une commande de billets de manière atomique grâce à une transaction. Elle commence par vérifier la disponibilité de chaque ticket demandé en consultant la base de données et en s'assurant qu'aucun ticket n'est déjà réservé et que l'événement associé n'est pas passé. Une fois ces vérifications effectuées, elle calcule le montant total de la commande en multipliant le nombre de tickets par leur prix unitaire. La fonction crée ensuite un enregistrement de commande dans la base avec le statut "en attente de paiement" et des métadonnées incluant les informations du client et un identifiant de transaction unique. Elle lie ensuite tous les tickets validés à cette nouvelle commande en mettant à jour leur statut et leur référence de commande. Enfin, elle retourne l'objet complet de la commande avec toutes les informations relatives à l'utilisateur, aux tickets et au paiement éventuel. Si une erreur survient à n'importe quelle étape du processus, la transaction est automatiquement annulée pour garantir qu'aucune donnée partielle ne soit enregistrée, assurant ainsi la cohérence et l'intégrité des données dans le système.

Version prisma :

```

1  async createOrder(data: OrderCreateInput): Promise<OrderWithRelations> {
2    const { userId, tickets, customerInfo } = data;
3    return await prisma.$transaction(async (tx) => {
4      let totalAmount = 0;
5      const ticketItems: Array<{
6        ticketId: string;
7        quantity: number;
8        unitPrice: number;
9      }> = [];
10     const reservedTickets: string[] = [];
11
12     try {
13       for (const item of tickets) {
14         const ticket = await tx.ticket.findUnique({
15           where: { id: item.ticketId },
16           include: {
17             event: true,
18             order: true
19           }
20         });
21
22         if (!ticket) {
23           throw new Error(`Ticket with ID ${item.ticketId} not found`);
24         }
25
26         if (ticket.orderId) {
27           throw new Error(`Ticket ${item.ticketId} is already reserved`);
28         }
29
30         if (ticket.event && new Date(ticket.event.date) < new Date()) {
31           throw new Error(`Event ${ticket.event.title} has already passed`);
32         }
33
34         const unitPrice = 50;
35
36         totalAmount += unitPrice * item.quantity;
37         ticketItems.push({
38           ticketId: item.ticketId,
39           quantity: item.quantity,
40           unitPrice
41         });
42
43         reservedTickets.push(item.ticketId);
44       }
45
46       const order = await tx.order.create({
47         data: {
48           userId,
49           status: OrderStatus.pending_payment,
50           totalPrice: totalAmount,
51           currency: 'EUR',
52           metadata: {
53             customerInfo,
54             ticketItems,
55             transactionId: `TXN_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`
56           }
57         }
58       });
59
60       for (const ticketId of reservedTickets) {
61         await tx.ticket.update({
62           where: { id: ticketId },
63           data: {
64             orderId: order.id,
65             status: 'pending'
66           }
67         });
68       }
69
70       return await tx.order.findUnique({
71         where: { id: order.id },
72         include: {
73           user: {
74             select: {
75               id: true,
76               name: true,
77               email: true
78             }
79           },
80           tickets: {
81             include: {
82               event: true
83             }
84           },
85           payment: true
86         }
87       }) as OrderWithRelations;
88
89     } catch (error) {
90       throw new Error(`Order creation failed: ${error instanceof Error ? error.message : 'Unknown error'}`);
91     }
92   });
93 }

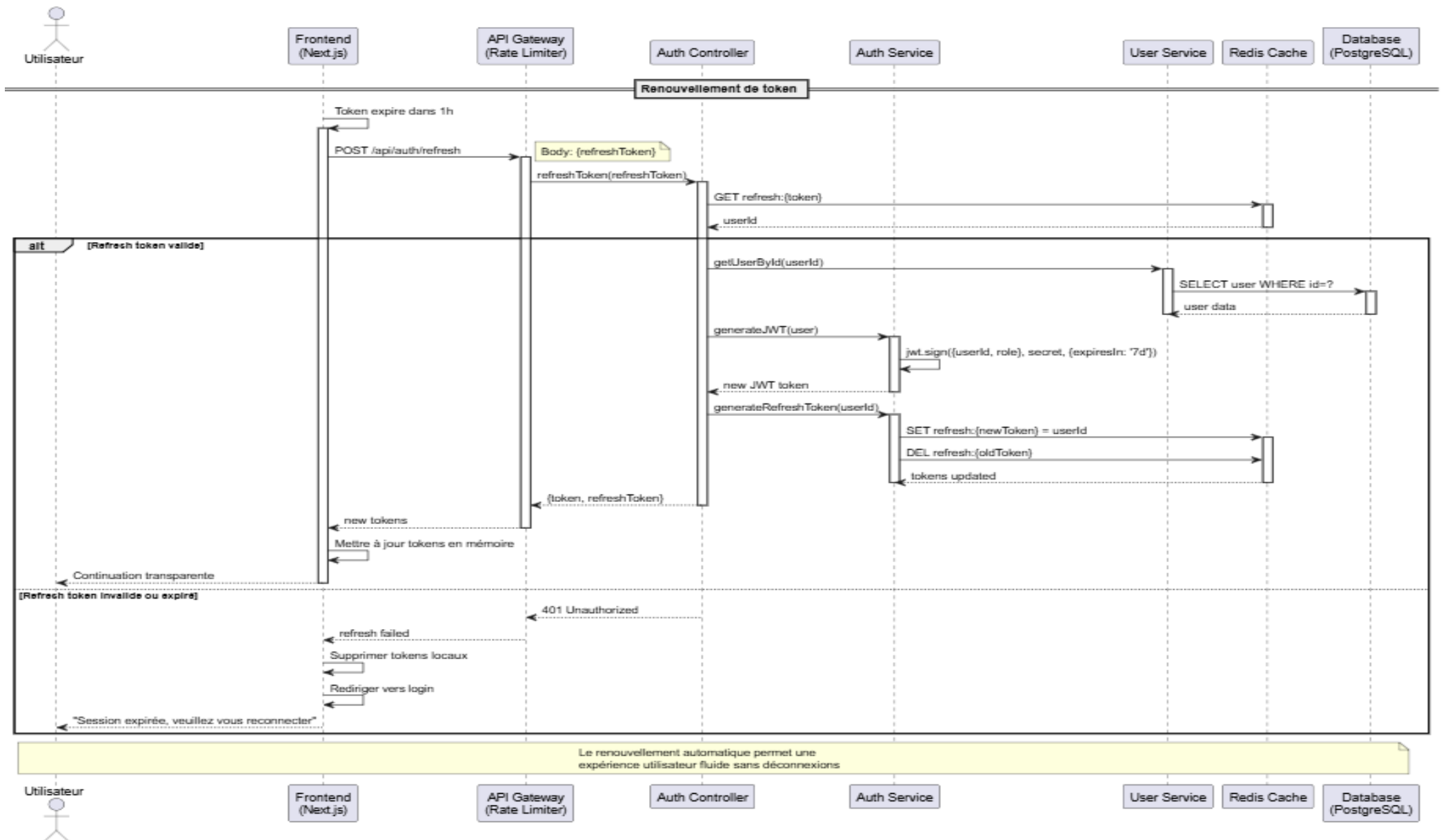
```

Version SQL :

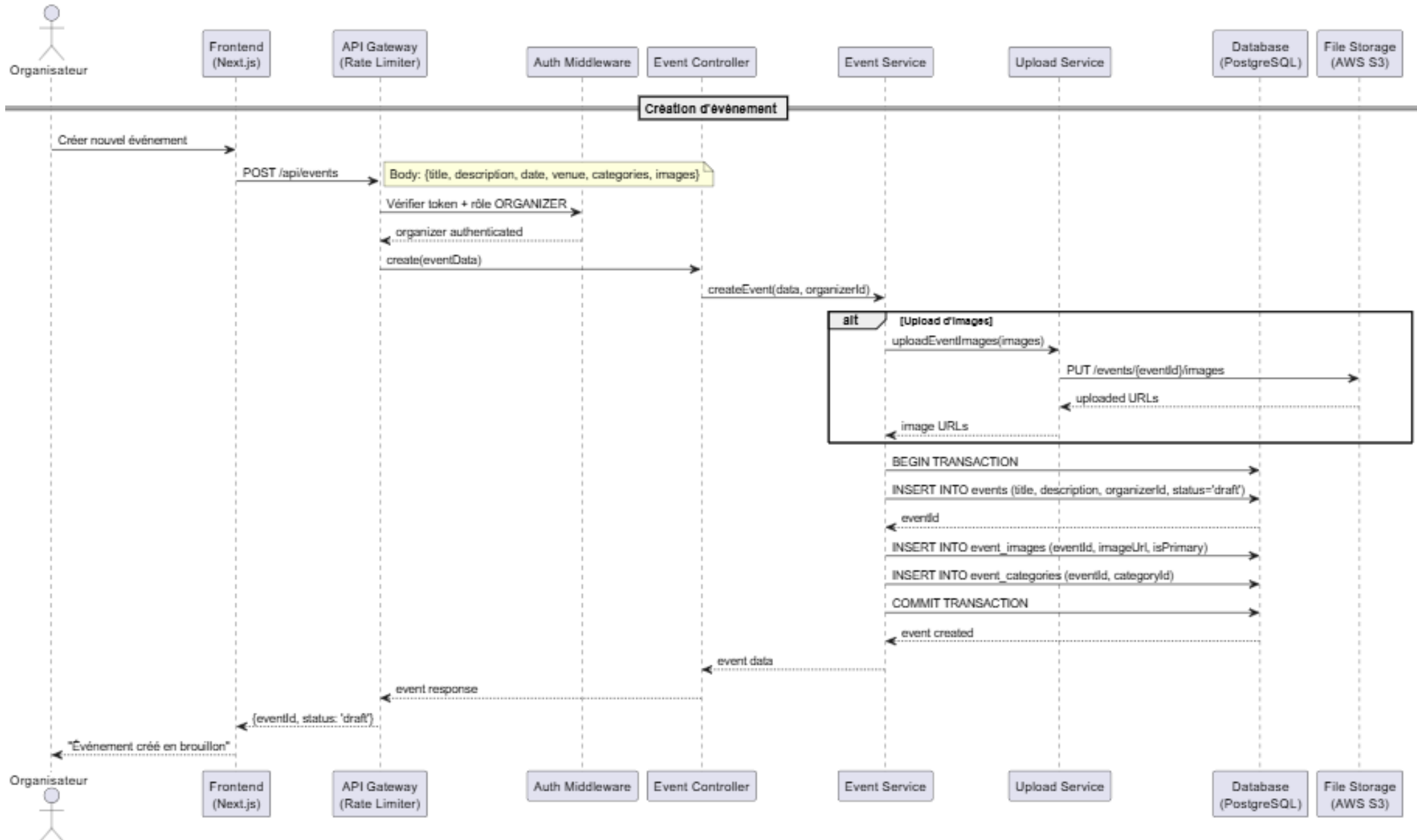
```
1 BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
2
3 SELECT t.id, t."eventId", t."orderId", t.status,
4        e.title AS event_title, e.date AS event_date
5 FROM "Ticket" t
6 INNER JOIN "Event" e ON t."eventId" = e.id
7 WHERE t.id = ANY($1)
8 FOR UPDATE NOWAIT;
9
10 DO $$
11 DECLARE
12     v_order_id UUID;
13     v_total_amount NUMERIC := 0;
14     v_ticket_count INT;
15     v_unit_price NUMERIC := 50;
16 BEGIN
17     SELECT COUNT(*) INTO v_ticket_count
18     FROM "Ticket" t
19     INNER JOIN "Event" e ON t."eventId" = e.id
20     WHERE t.id = ANY($1)
21           AND t."orderId" IS NULL
22           AND e.date >= NOW();
23
24     IF v_ticket_count < ARRAY_LENGTH($1, 1) THEN
25         RAISE EXCEPTION 'One or more tickets are not available';
26     END IF;
27
28     v_total_amount := v_ticket_count * v_unit_price;
29
30     INSERT INTO "Order" (
31         id, "userId", status, "totalPrice",
32         currency, metadata, "createdAt", "updatedAt"
33     ) VALUES (
34         gen_random_uuid(), $2, 'pending_payment', v_total_amount,
35         'EUR',
36         jsonb_build_object(
37             'customerInfo', $3,
38             'ticketItems', '[]'::JSONB,
39             'transactionId',
40             CONCAT('TXN_', EXTRACT(EPOCH FROM NOW())::TEXT, '_',
41                   substr(md5(random()::text), 1, 9))
42         ),
43         NOW(), NOW()
44     ) RETURNING id INTO v_order_id;
45
46     UPDATE "Ticket"
47     SET "orderId" = v_order_id, status = 'pending', "updatedAt" = NOW()
48     WHERE id = ANY($1) AND "orderId" IS NULL;
49
50     IF NOT FOUND THEN
51         RAISE EXCEPTION 'Failed to reserve tickets';
52     END IF;
53 END $$;
54
55 SELECT o.id, o.status, o."totalPrice", o.currency, o."createdAt",
56        json_build_object('id', u.id, 'name', u.name,
57                          'email', u.email) AS user,
58        (SELECT json_agg(json_build_object(
59            'id', t.id, 'code', t.code, 'status', t.status,
60            'event', json_build_object('id', e.id, 'title', e.title,
61                                      'date', e.date, 'location', e.location)
62        ))
63     FROM "Ticket" t
64     INNER JOIN "Event" e ON t."eventId" = e.id
65     WHERE t."orderId" = o.id
66     ) AS tickets
67 FROM "Order" o
68 INNER JOIN "User" u ON o."userId" = u.id
69 WHERE o.id = (SELECT id FROM "Order" ORDER BY "createdAt" DESC LIMIT 1);
70
71 COMMIT;
```

Diagrammes de séquence :

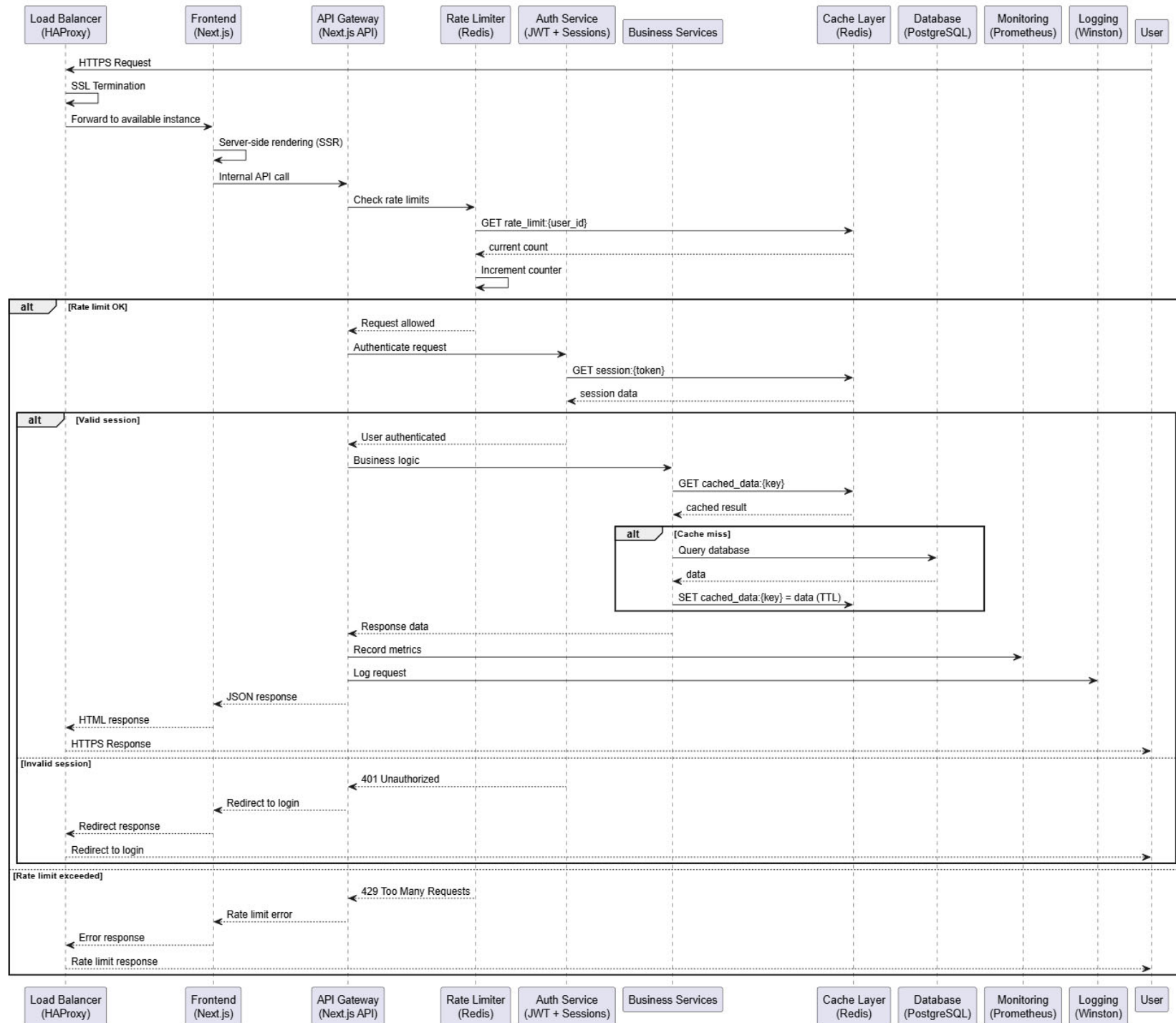
Processus de renouvellement de token JWT



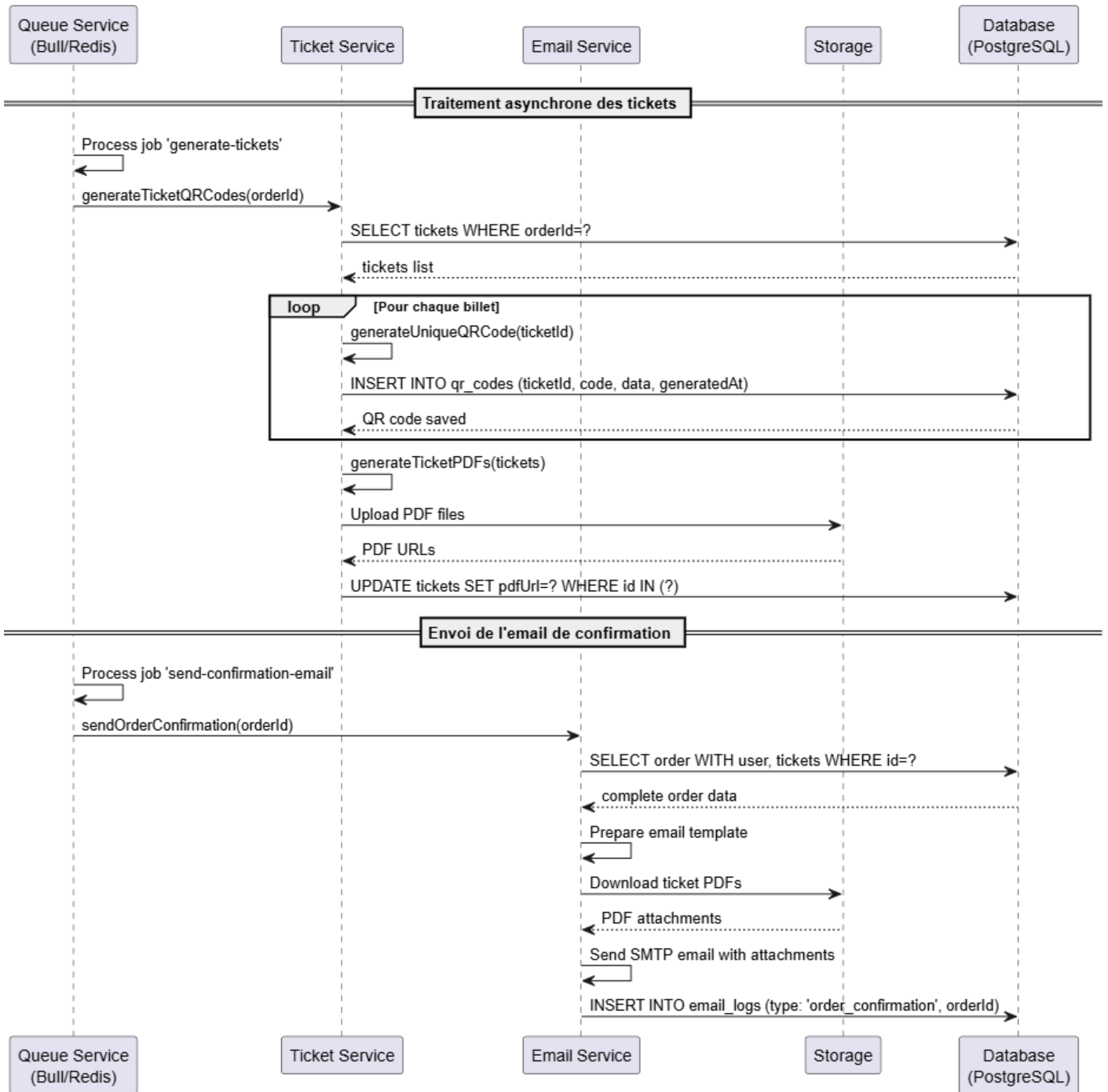
Création d'évènement par l'organisateur



Flux de requête utilisateur classique



Génération des billets et envoi d'emails de confirmation



8. Éléments de sécurité de l'application

La sécurité de l'application M&C Society a été intégrée dès la phase de conception et repose sur une approche globale couvrant la protection des données, l'authentification, la prévention des attaques et le contrôle des accès. L'objectif est d'assurer la confidentialité, l'intégrité et la disponibilité des informations tout en respectant les normes et bonnes pratiques en matière de cybersécurité.

8.1 Protection des mots de passe

La protection des mots de passe constitue un élément fondamental de la sécurité de l'application :

- **Hachage robuste** : tous les mots de passe sont hachés avec l'algorithme bcrypt, configuré pour 10 à 12 rounds selon les besoins de performance et de sécurité.
- **Politique de mot de passe fort** : les mots de passe doivent comporter au minimum 8 caractères et inclure une majuscule, une minuscule, un chiffre et un caractère spécial, afin de limiter les risques de compromission par force brute ou dictionnaire.
- **Gestion de l'historique** : chaque modification de mot de passe est suivie via le champ `passwordChangedAt`, ce qui permet de détecter et de prévenir la réutilisation de mots de passe anciens ou compromis.
- **Aucune conservation en clair** : les mots de passe ne sont jamais stockés en clair dans la base de données, assurant une protection maximale en cas de fuite de données.
- **Suivi et audit** : toutes les opérations relatives aux mots de passe (création, modification, réinitialisation) sont enregistrées dans les journaux de sécurité, permettant une traçabilité complète.

8.2 Authentification sécurisée par JWT

L'authentification repose sur l'usage de **JSON Web Tokens (JWT)**, avec des mécanismes robustes pour la gestion des sessions et le contrôle d'accès :

- **Séparation des tokens** : l'application utilise des tokens d'accès de courte durée (15 minutes) et des tokens de rafraîchissement d'une durée plus longue (7 jours), afin de limiter l'exposition en cas de compromission.
- **Stockage sécurisé des sessions** : toutes les sessions utilisateur sont conservées dans la base PostgreSQL, permettant le suivi et la révocation centralisée en cas de besoin.
- **Invalidation automatique** : les sessions expirées sont supprimées automatiquement, et un administrateur peut révoquer manuellement toute session suspecte.
- **Contrôle d'accès basé sur les rôles (RBAC)** : l'application distingue différents niveaux d'accès (VISITOR < USER < ORGANIZER < ADMIN), garantissant que chaque utilisateur ne puisse exécuter que les actions autorisées selon son rôle.
- **Protection contre le détournement** : les tokens sont signés et vérifiés côté serveur, et les mécanismes de renouvellement sont sécurisés pour éviter l'usurpation d'identité.

8.3 Protection contre les injections et XSS

M&C Society met en œuvre des mécanismes stricts pour prévenir les attaques classiques telles que les injections SQL ou les scripts intersites (XSS) :

- **Validation stricte des entrées** : toutes les données fournies par les utilisateurs sont validées à l'aide de schémas robustes (via Zod), afin de garantir leur conformité et d'empêcher l'injection de code malveillant.
- **Utilisation de Prisma ORM** : le recours à un ORM typé et sécurisé permet de générer des requêtes préparées automatiquement, limitant fortement le risque d'injection SQL.
- **Filtrage des réponses** : les champs sensibles, tels que les mots de passe ou les informations financières, sont systématiquement exclus des réponses aux clients.
- **Headers de sécurité et CSP** : l'application applique une Content Security Policy (CSP) stricte et des headers HTTP de sécurité (X-Content-Type-Options, X-Frame-Options, Referrer-Policy) pour limiter le chargement de ressources non autorisées et réduire le risque d'attaques XSS.
- **Protection CSRF** : des jetons synchronisés sont utilisés pour protéger les formulaires critiques contre les attaques de falsification de requêtes intersites.

8.4 Gestion des accès CORS

La configuration Cross-Origin Resource Sharing (CORS) est optimisée pour limiter les échanges aux domaines autorisés :

- **Domaines autorisés** : seuls les domaines explicitement configurés peuvent accéder à l'API, réduisant le risque de requêtes non autorisées depuis des sites tiers.
- **Configuration par environnement** : les règles CORS sont adaptées selon l'environnement (développement, test, production) pour garantir flexibilité et sécurité.
- **Restriction des méthodes et headers** : les méthodes HTTP autorisées et les headers exposés sont strictement limités aux besoins fonctionnels, réduisant la surface d'attaque et les possibilités d'exploitation abusive.

8.5 Conformité RGPD

- L'application **M&C Society** respecte le Règlement Général sur la Protection des Données (RGPD) et met en œuvre les principes de transparence, de minimisation et de sécurité des données. Les mesures suivantes sont intégrées au projet :
- **Droit d'accès** : chaque utilisateur peut consulter l'ensemble de ses données personnelles à partir de son espace sécurisé.
- **Droit de portabilité** : un export complet et structuré de ses données est disponible au format **JSON** ou **CSV**.
- **Droit d'effacement (droit à l'oubli)** : l'utilisateur peut demander la suppression de ses données. La suppression est réalisée de manière sécurisée, avec prise en compte des contraintes légales (ex. conservation des données comptables pendant 10 ans).
- **Droit de rectification et d'opposition** : une API et une interface utilisateur permettent la modification de ses données, ainsi que la gestion des préférences (notifications, communication marketing).

- **Traçabilité et journalisation** : toutes les actions liées aux données personnelles (consultation, modification, suppression, export) sont consignées dans un **journal d'audit** horodaté.
- **Durée de conservation des données** :
 - Comptes utilisateurs : conservés tant que l'utilisateur dispose d'un compte actif. En cas de suppression ou d'inactivité prolongée (3 ans), les données sont anonymisées ou effacées sous 30 jours.
 - Commandes, billets et factures : conservés **10 ans** conformément aux obligations légales et fiscales.
 - Données bancaires : aucune donnée sensible n'est stockée. Les paiements sont intégralement gérés par le prestataire Stripe. Seuls les identifiants de transaction nécessaires sont conservés pendant 10 ans.
 - Logs techniques et de sécurité : conservés **1 an maximum** pour assurer la traçabilité et la détection des anomalies.
 - Données marketing et prospection (newsletter, préférences, historique de navigation) : conservées **3 ans après le dernier contact** ou la dernière activité de l'utilisateur.
- **Suppression et anonymisation** : à l'issue des délais de conservation, les données sont supprimées de manière sécurisée ou anonymisées afin de ne conserver que des informations statistiques.

8.6 Mesures complémentaires

- En complément des mécanismes de protection intégrés (authentification, RGPD, sécurisation des accès), l'application **M&C Society** met en œuvre plusieurs mesures supplémentaires afin de renforcer la sécurité globale :
- **Rate limiting** : un système de limitation des requêtes est en place afin de prévenir les attaques par force brute et les abus (par exemple sur les formulaires de connexion ou de paiement). Ce mécanisme bloque ou ralentit automatiquement les requêtes excessives provenant d'une même source.

- **Monitoring et journalisation de sécurité** : les événements critiques (connexions suspectes, tentatives d'intrusion, échecs répétés d'authentification) sont enregistrés dans un **journal de sécurité**. Des alertes automatiques sont générées en cas de comportement anormal afin de permettre une réaction rapide des administrateurs.
- **Cookies sécurisés** : les cookies de session et d'authentification sont configurés avec les attributs **HTTPOnly** (inaccessibles depuis JavaScript), **Secure** (transmis uniquement via HTTPS) et **SameSite** (limitation du partage inter-domaines) afin de réduire les risques de vol ou de détournement de session.
- **Headers de sécurité** : l'application applique des en-têtes de sécurité conformes aux bonnes pratiques :
 - **HSTS (HTTP Strict Transport Security)** : impose l'utilisation du protocole HTTPS pour toutes les communications.
 - **CSP (Content Security Policy)** : limite le chargement des scripts, styles et ressources aux seules sources autorisées, afin de réduire les risques d'attaques XSS.
 - **Protections CSRF** : mises en place pour éviter la falsification de requêtes intersites (ex. utilisation de jetons synchronisés).
 - **Autres en-têtes complémentaires** : configuration des politiques X-Frame-Options, X-Content-Type-Options et Referrer-Policy pour limiter les risques d'injection et de fuite d'informations.

Ces mesures, combinées aux protections natives de la plateforme, contribuent à assurer un haut niveau de sécurité pour les utilisateurs et les organisateurs d'événements.

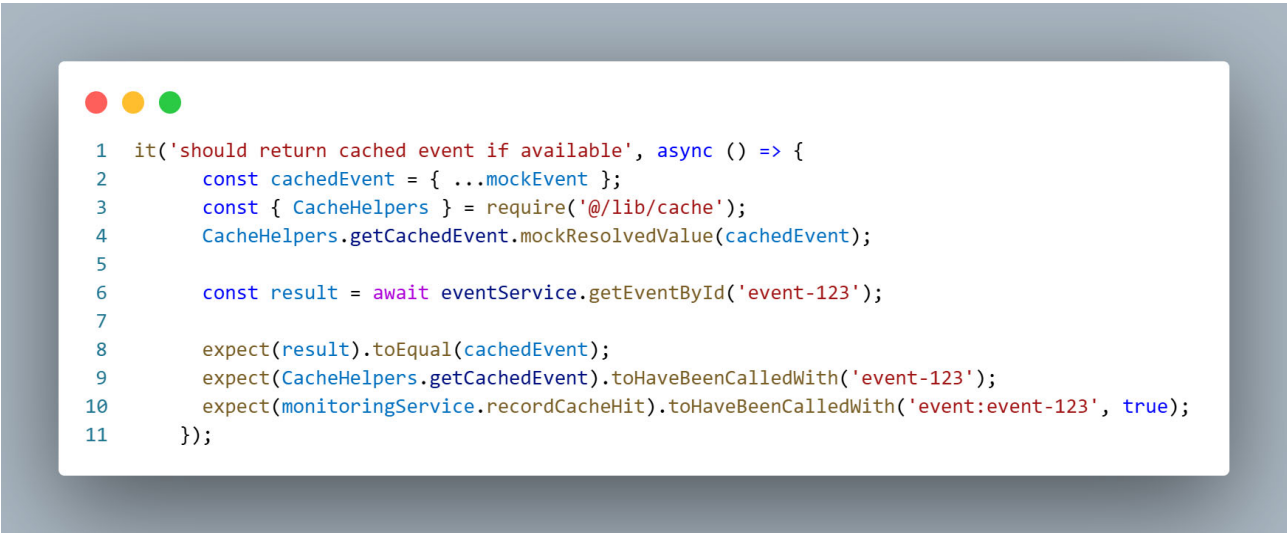
9. Plan de tests

La qualité et la fiabilité de l'application M&C Society ont été assurées par la mise en place d'une stratégie complète couvrant l'ensemble du cycle de vie du développement.

9.1 Stratégie de tests

L'approche adoptée repose sur une pyramide de tests :

- **Tests unitaires (≈70 %)** : validation de la logique métier isolée (services, contrôleurs, utilitaires).



```
1  it('should return cached event if available', async () => {
2      const cachedEvent = { ...mockEvent };
3      const { CacheHelpers } = require('@/lib/cache');
4      CacheHelpers.getCachedEvent.mockResolvedValue(cachedEvent);
5
6      const result = await eventService.getEventById('event-123');
7
8      expect(result).toEqual(cachedEvent);
9      expect(CacheHelpers.getCachedEvent).toHaveBeenCalledWith('event-123');
10     expect(monitoringService.recordCacheHit).toHaveBeenCalledWith('event:event-123', true);
11 });
```

Ce test vérifie le mécanisme de cache Redis pour optimiser les performances. Lorsqu'un utilisateur consulte un événement déjà récemment chargé, le système récupère les données depuis le cache au lieu d'interroger la base de données PostgreSQL. (réduit la latence, enregistre les métriques de cache pour le monitoring, allège la charge sur la BDD en production, permet un lazy loading)



```

1  it('should fetch from database and cache if not in cache', async () => {
2      const { CacheHelpers } = require('@lib/cache');
3      CacheHelpers.getCachedEvent.mockResolvedValue(null);
4
5      const prismaMock = prisma as any;
6      prismaMock.event.findUnique.mockResolvedValue(mockEvent);
7
8      const result = await eventService.getEventById('event-123');
9
10     expect(result).toEqual(mockEvent);
11     expect(CacheHelpers.getCachedEvent).toHaveBeenCalledWith('event-123');
12     expect(CacheHelpers.cacheEvent).toHaveBeenCalledWith('event-123', mockEvent);
13     expect(monitoringService.recordCacheHit).toHaveBeenCalledWith('event:event-123', false);
14 });

```

Ce test valide la stratégie de fallback : si l'événement n'est pas dans le cache (cache miss), le service interroge PostgreSQL via Prisma, puis met automatiquement en cache le résultat pour les prochaines requêtes. (le système fonctionne même si le cache est vide, améliore progressivement les performances, trace les cache miss pour le monitoring, garantit des données "fraîches")



```

1  it('should handle database connection errors', async () => {
2      const prismaMock = prisma as any;
3      prismaMock.event.findUnique.mockRejectedValue(new Error('Database connection failed'));
4      const { CacheHelpers } = require('@lib/cache');
5      CacheHelpers.getCachedEvent.mockResolvedValue(null);
6
7      await expect(eventService.getEventById('event-123')).rejects.toThrow('Database connection failed');
8  });

```

Ce test simule une panne de connexion à PostgreSQL (réseau, surcharge, maintenance) et vérifie que l'erreur est correctement propagée au niveau supérieur pour être gérée par le middleware d'erreur.

- **Tests d'intégration (≈20 %)** : validation des interactions entre les composants (base de données, services externes comme Stripe et SMTP).



```
1  it('should handle payment failure and rollback order', async () => {
2      const mockOrder = {
3          id: 'order-fail-123',
4          userId: mockUser.id,
5          totalAmount: 50.0,
6          status: OrderStatus.PENDING,
7          currency: 'EUR',
8          createdAt: new Date(),
9          updatedAt: new Date(),
10     };
11
12     mockPrisma.order.create.mockResolvedValue(mockOrder);
13     mockPrisma.payment.create.mockRejectedValue(new Error('Payment failed'));
14     mockPrisma.order.update.mockResolvedValue({
15         ...mockOrder,
16         status: OrderStatus.CANCELLED,
17     });
18
19     // Create order
20     const order = await mockPrisma.order.create({
21         data: {
22             userId: mockUser.id,
23             totalAmount: 50.0,
24             status: OrderStatus.PENDING,
25             currency: 'EUR',
26         },
27     });
28
29     // Try to process payment
30     let paymentError;
31     try {
32         await mockPrisma.payment.create({
33             data: {
34                 orderId: order.id,
35                 amount: order.totalAmount,
36                 currency: 'EUR',
37                 paymentMethod: 'CARD',
38                 paymentStatus: 'PENDING',
39                 transactionId: 'pi_fail',
40                 paymentDate: new Date(),
41             },
42         });
43     } catch (error: any) {
44         paymentError = error;
45     }
46
47     expect(paymentError).toBeDefined();
48     expect(paymentError.message).toBe('Payment failed');
49
50     // Rollback order
51     const cancelledOrder = await mockPrisma.order.update({
52         where: { id: order.id },
53         data: { status: OrderStatus.CANCELLED },
54     });
55
56     expect(cancelledOrder.status).toBe(OrderStatus.CANCELLED);
57 });
```

Ce test simule un échec de paiement Stripe et vérifie que la commande est automatiquement annulée pour maintenir la cohérence. Cela évite d'avoir des commandes fantômes en base et libère immédiatement les places pour d'autres acheteurs.

```
1  it('should handle concurrent ticket purchases', async () => {
2    const eventWithLimitedTickets = {
3      ...mockEvent,
4      availableTickets: 1,
5    };
6
7    mockPrisma.event.findUnique.mockResolvedValue(eventWithLimitedTickets);
8
9    // First transaction succeeds
10   mockPrisma.$transaction.mockResolvedValueOnce({
11     id: 'order-1',
12     status: OrderStatus.PENDING,
13   });
14
15   // Second transaction fails due to sold out
16   mockPrisma.$transaction.mockRejectedValueOnce(
17     new Error('Not enough tickets available')
18   );
19
20   const result1 = await mockPrisma.$transaction([]);
21   expect(result1).toBeDefined();
22
23   let error;
24   try {
25     await mockPrisma.$transaction([]);
26   } catch (e: any) {
27     error = e;
28   }
29
30   expect(error).toBeDefined();
31   expect(error.message).toContain('Not enough tickets available');
32 });
```

Ce test reproduit la situation où deux utilisateurs tentent d'acheter le dernier billet simultanément. Grâce aux transactions atomiques Prisma, un seul achat réussit tandis

que l'autre reçoit une erreur explicite, garantissant qu'on ne vend jamais plus de billets que la capacité disponible.



```
1  it('should prevent double ticket validation', async () => {
2      const usedTicket = {
3          id: 'ticket-used',
4          code: 'TKT-USED-123',
5          status: TicketStatus.USED,
6          eventId: 'event-123',
7          userId: 'user-123',
8          orderId: 'order-123',
9          seatNumber: 'A1',
10         purchasedAt: new Date(),
11         createdAt: new Date(),
12     };
13
14     mockPrisma.ticket.findUnique.mockResolvedValue(usedTicket);
15
16     const ticket = await mockPrisma.ticket.findUnique({
17         where: { id: 'ticket-used' },
18     });
19
20     expect(ticket?.status).toBe(TicketStatus.USED);
21
22     // Should not allow re-validation
23     const canValidate = ticket!.status === TicketStatus.VALID;
24     expect(canValidate).toBe(false);
25 });
```

Ce test vérifie qu'un billet déjà scanné à l'entrée ne peut pas être réutilisé. Une fois le statut passé à USED, toute tentative de validation ultérieure doit échouer, empêchant ainsi la fraude et l'usage multiple d'un même billet.

- **Tests end-to-end (≈10 %)** : validation des parcours utilisateurs complets (visiteur, organisateur, administrateur).



```
1 test('complete flow - register → login → purchase ticket', async ({ page }) => {
2   await test.step('Registration', async () => {
3     await page.goto('/register');
4
5     await page.fill('[name="name"]', 'Test User');
6     await page.fill('[name="email"]', testEmail);
7     await page.fill('[name="password"]', testPassword);
8     await page.fill('[name="confirmPassword"]', testPassword);
9
10    await page.click('button[type="submit"]');
11
12    await expect(page).toHaveURL(/\/(dashboard|login)/, { timeout: 10000 });
13  });
14
15  await test.step('Login', async () => {
16    if (page.url().includes('/login')) {
17      await page.fill('[name="email"]', testEmail);
18      await page.fill('[name="password"]', testPassword);
19      await page.click('button[type="submit"]');
20
21      await expect(page).toHaveURL(/\/dashboard/, { timeout: 10000 });
22    }
23  });
24
25  await test.step('Event selection', async () => {
26    await page.goto('/events');
27
28    await page.waitForSelector('.event-card, [data-testid="event-card"]', { timeout: 10000 });
29
30    const firstEvent = page.locator('.event-card, [data-testid="event-card"]').first();
31    await firstEvent.click();
32
33    await expect(page).toHaveURL(/\/events\/[^\/]+/, { timeout: 10000 });
34  });
35
36  await test.step('Ticket purchase', async () => {
37    const buyButton = page.locator(
38      'button:has-text("Acheter"), button:has-text("Buy"), [data-testid="buy-ticket"]'
39    ).first();
40
41    await expect(buyButton).toBeVisible({ timeout: 10000 });
42    await buyButton.click();
43
44    await expect(page).toHaveURL(/\/(checkout|payment|order)/, { timeout: 15000 });
45  });
46
47  await test.step('Stripe test payment', async () => {
48    await page.waitForSelector(
49      'iframe[name^="__privateStripeFrame"], [data-testid="payment-form"]',
50      { timeout: 15000 }
51    );
52  });
53
54  await test.step('Verify ticket received', async () => {
55    await page.goto('/tickets');
56
57    const ticketItem = page.locator('.ticket-item, [data-testid="ticket-card"]').first();
58    await expect(ticketItem).toBeVisible({ timeout: 10000 });
59
60    const qrCode = ticketItem.locator('img[alt*="QR"]', canvas, [data-testid="qr-code"]);
61    await expect(qrCode).toBeVisible();
62  });
63 });
```


Ce test automatise le parcours utilisateur complet dans un navigateur Chrome via Playwright. Il crée un compte, se connecte, navigue dans les événements, achète un billet via Stripe et vérifie la présence du QR code dans la section billets. C'est une validation end-to-end de toute la stack technique.



```
1 test('redirect to login if not authenticated', async ({ page }) => {
2     await page.goto('/dashboard');
3
4     await expect(page).toHaveURL(/\\/login/, { timeout: 10000 });
5 });
```

Ce test vérifie le middleware de sécurité qui redirige automatiquement les utilisateurs non authentifiés vers la page de connexion lorsqu'ils tentent d'accéder au dashboard ou à toute autre page protégée.



```
1 test('login with invalid credentials shows error', async ({ page }) => {
2     await page.goto('/login');
3
4     await page.fill('[name="email"]', 'wrong@example.com');
5     await page.fill('[name="password"]', 'wrongpassword');
6     await page.click('button[type="submit"]');
7
8     await expect(page.locator('text=/invalid|incorrect|wrong/i')).toBeVisible({ timeout: 5000 });
9 });
```

Ce test valide que l'interface affiche un message d'erreur clair et visible lorsqu'un utilisateur tente de se connecter avec des identifiants incorrects, garantissant un bon feedback UX.

9.2 Organisation et outillage

- **Framework de tests** : Jest avec support TypeScript (ts-jest).
- **Mocking** : Prisma (base de données), Stripe (paiements), Email (notifications), fichiers statiques.
- **Validation des entrées** : via schémas Zod.
- **Couverture** : rapports générés (Istanbul/LCOV) avec un objectif minimal de **80 % global**, et **95 % sur les modules critiques** (paiements, sécurité).
- **CI/CD** : exécution automatisée des tests avec reporting (console, JSON, HTML).

9.3 Types de tests effectués

- **Unitaires** :
 - AuthService (hachage, validation des mots de passe, JWT).
 - TicketService (génération, validation des billets).
 - PaymentService (paiement et remboursement via Stripe).
 - EventService (gestion des événements et réservations).
 - QRCodeService (génération, rotation des QR codes toutes les 12h).
 - CacheService (stratégies de mise en cache Redis, invalidation).
 - Middlewares (rate limiting, authentification, CORS).

- **Intégration :**

- Base de données PostgreSQL via Prisma (CRUD, migrations, transactions).
- Services externes : Stripe (paiements en mode test), SMTP (envoi d'e-mails), Redis (cache).
- Workflows complets :
 1. Processus de réservation (parcours d'achat complet).
 2. Rollback transactionnel en cas d'échec paiement.
 3. Gestion de la concurrence (race conditions, derniers billets).
 4. Création d'événement, administration.
- Génération et validation des QR codes avec rotation automatique.

- **API :**

- Routes publiques (consultation événements, inscription, connexion).
- Routes protégées (achat de billets, gestion de profil).
- Routes administrateur (gestion utilisateurs, modération événements, configuration système).
- Webhooks Stripe (traitement des événements de paiement asynchrones).
- Gestion d'erreurs et codes HTTP appropriés (400, 401, 403, 500).

- **End-to-End (E2E) :**

- **Visiteur** : recherche et achat de billets, réception d'e-mails de confirmation.
- **Organisateur** : création et gestion d'événements, suivi des ventes.

- **Administrateur** : modération, gestion des utilisateurs et statistiques.
- Scan de QR code à l'entrée (validation et détection de doublons).
- **Performance et charge** :
 - Tests de montée en charge (ventes flash, événements populaires).
 - Optimisation du cache Redis (temps de réponse < 100ms).
 - Tests de stress sur les transactions concurrentes.
- **Régression et snapshots** :
 - Tests de snapshots pour les templates d'e-mails.
 - Vérification de non-régression des composants UI critiques.
- **Sécurité et conformité** :
 - Tests anti-injections SQL et XSS.
 - Vérification des mécanismes CSRF et authentification JWT.
 - Validation RGPD (export/suppression des données).
 - Conformité PCI DSS pour les flux de paiement.
 - Tests d'accessibilité (A11Y).

9.4 Maintenance et évolutions

- Les tests sont organisés de façon modulaire et maintenus en parallèle du code.
- Des **scripts automatisés** facilitent l'exécution en local, en CI/CD et le seed de données de test.
- Les tests sont révisés régulièrement pour supprimer les doublons, renforcer la couverture et documenter les nouvelles fonctionnalités.
- Les métriques de performance et de sécurité sont surveillées (temps d'exécution < 30s pour la suite complète, monitoring mémoire).

10. Jeu d'essai de la fonctionnalité la plus représentative

Fonctionnalité testée : Achat de billets multi-événements avec sécurité avancée, gestion de stock, paiement sécurisé, génération de billets et notifications automatiques.

Objectif : Valider le parcours complet de l'utilisateur, de l'authentification à la validation des billets, tout en garantissant la sécurité, la fiabilité et la traçabilité.

Scénario de test :

1. **Authentification sécurisée :** connexion avec JWT, suivi des tentatives (LoginAttempt), détection d'IP suspecte, rate limiting.
2. **Sélection multi-billets :** choix de 2 billets VIP et 1 billet Standard pour deux événements distincts, avec vérification en temps réel de la disponibilité.
3. **Création de commande :** orchestration via OrchestrationService, validation des stocks et génération sécurisée du PaymentIntent Stripe.
4. **Paiement sécurisé :** validation 3D Secure, traitement des webhooks Stripe avec signature cryptographique et transactions atomiques.
5. **Génération des billets :** création de QR codes uniques avec rotation programmée, chiffrement AES-256.
6. **Notifications automatiques :** envoi d'emails de confirmation et de billets, notification in-app.
7. **Sécurité et tests avancés :** prévention de la réutilisation de QR codes, validation de checksum et gestion des billets expirés.
8. **Validation à l'entrée :** scan QR code via API, vérification multicouche (format, signature, expiration, double usage), mise à jour du statut isScanned.
9. **Audit et surveillance :** enregistrement dans AuditLog, mise à jour des sessions, suivi des métriques en temps réel.

Résultats attendus :

- Commande status: 'paid', totalPrice: 95,50€
- Paiement confirmé (paymentStatus: 'COMPLETED')
- 3 billets avec QR codes uniques et rotation programmée
- 2 emails envoyés et notification in-app
- Traçabilité complète et audit conforme GDPR
- Stock et transactions sécurisés, processus atomique

Conclusion : Ce scénario illustre le fonctionnement global du système et permet de vérifier l'ensemble des fonctionnalités critiques de manière sécurisée et automatisée.

11. Veille sur les vulnérabilités de sécurité

Une veille régulière a été effectuée autour de ces sujets :

- Bonnes pratiques d'**authentification** en Next.js
- Failles fréquentes (XSS, CSRF, injections SQL)
- Pratiques RGPD : données personnelles, droit à l'oubli
- Utilisation des alertes GitHub sur les dépendances
- Mise à jour des packages critiques (ex. Prisma, JWT)

Sources utilisées : owasp.org, blog.risingstack.com, GitHub Security Advisories, X (twitter), Instagram, LinkedIn.